Unit-II: Principles of Parallel and Distributed Computing

Unit-II: Concepts

- Principles of Parallel and Distributed Computing:
- Eras of computing,
- Parallel vs. Distributed Computing,
- Elements of Parallel Computing,
- Elements of Distributed Computing,
- Technologies for Distributed Computing.

Concepts

- 2.1 Eras of Computing
- 2.2 Parallel vs. Distributed Computing
- 2.3 Elements of Parallel Computing
  - 2.3.1 What is Parallel Processing?
  - 2.3.2 Hardware Architectures for Parallel Processing
  - 2.3.3 Approaches to Parallel Programming
  - 2.3.4 Levels of Parallelism
  - 2.3.5 Laws of Caution
- 2.4 Elements of Distributed Computing
  - 2.4.1 General Concepts and Defi nitions
  - 2.4.2 Components of a Distributed System
  - 2.4.3 Architectural Styles for Distributed Computing
  - 2.4.4 Models for Inter-Process Communication
- 2.5 Technologies for Distributed Computing
  - 2.5.1 Remote Procedure Call
  - 2.5.2 Distributed Object Frameworks
  - 2.5.3 Service Oriented Computing

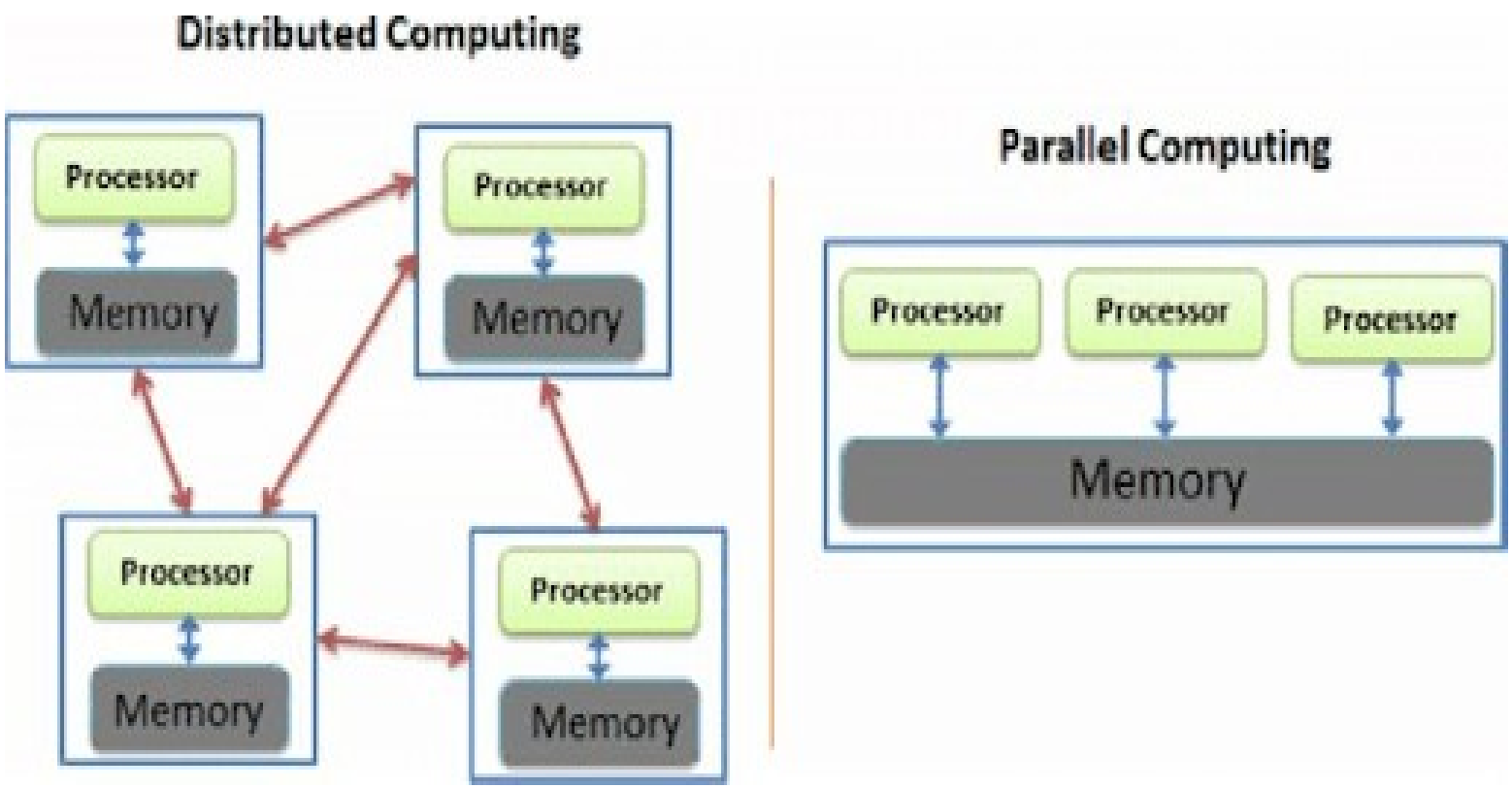
Introduction

- Cloud computing is a new technological trend that supports better utilization of IT infrastructures, services, and applications.
- Adopts a service delivery model based on a **pay-per-use** approach, in which users do not own infrastructure, platform, or applications but use them for the time they need them.
- IT assets are **owned and maintained by service providers** who make them accessible through the Internet.
- Parallel and distributed computing serve as foundations for building cloud computing systems and applications.

Principles of Parallel and Distributed Computing

- Parallel and distributed computing **are important technologies** that have key **differences in their primary function.**
- **Both** parallel and distributed computing have been around for a **long time** and both have **contributed greatly** to the **improvement of computing processes**. However, they have key differences in their primary function.
- Parallel computing, **also known as parallel processing**, speeds up a computational task by **dividing it into smaller jobs** across multiple processors inside one computer.
- Distributed computing, on the other hand, uses a distributed system, such as **the internet**, to **increase the available computing power** and enable larger, more complex tasks to be executed across multiple machines.

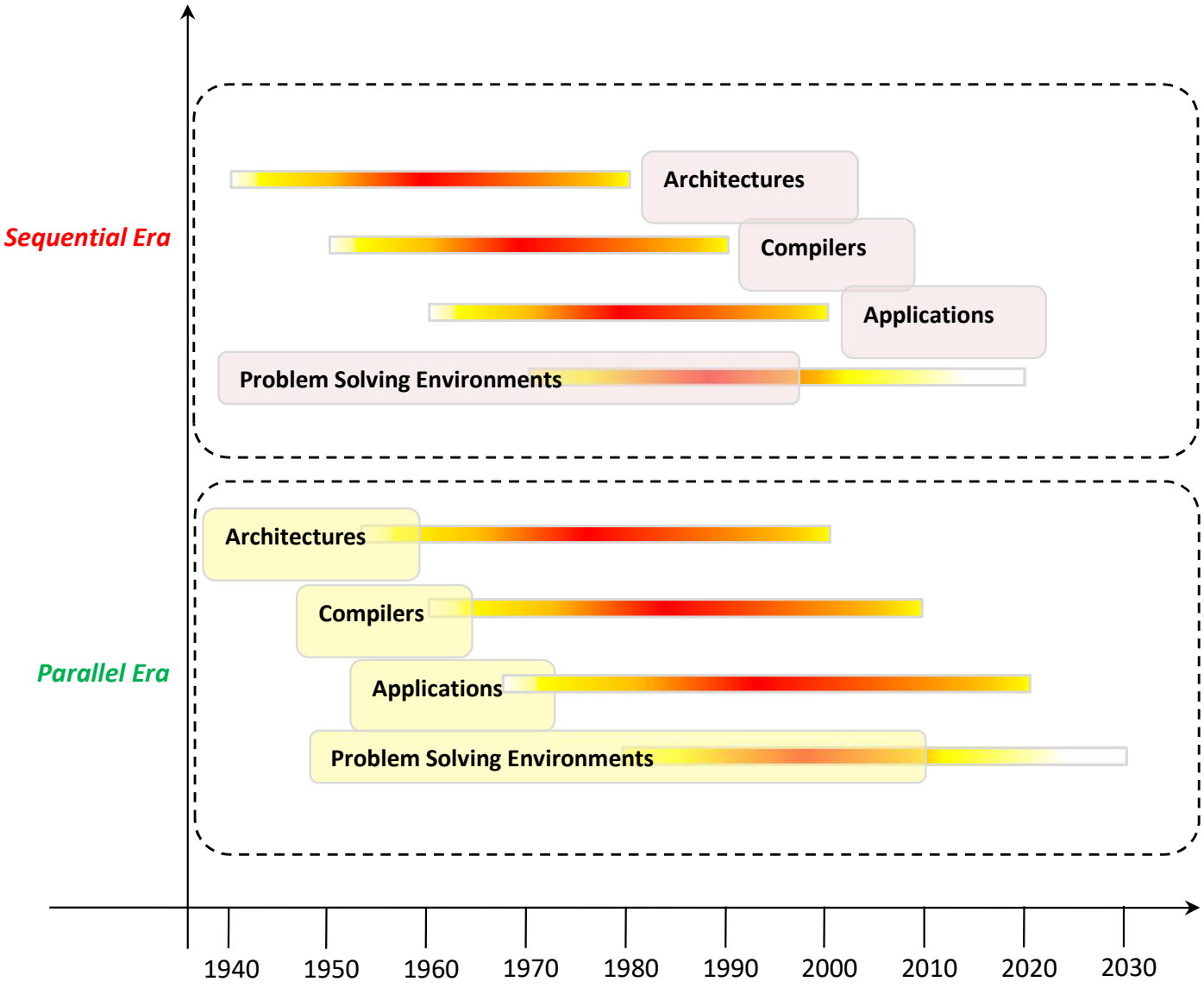
Principles of Parallel and Distributed Computing



Eras of Computing

- Two fundamental and dominant models of computing are ***sequential*** and ***parallel***.
  - The sequential era began in the 1940s, and Parallel(and distributed) computing era followed it within a decade.
- Four key elements of computing developed during three eras are
  - Architecture
  - Compilers
  - Applications
  - Problem solving environments
- The computing era started with development in ***hardware architectures***, which actually enabled the creation of ***system software*** – particularly in the area of **compilers and operating systems** – which support the management of such systems and the development of ***applications***.

Eras of Computing



Eras of Computing

- The development of **applications and systems** are the **major element** of interest and it comes to consolidation when **problem solving environments are designed and introduced to facilitate and empower engineers**.
- This is when the paradigm characterizing the **computing achieves** maturity and becomes a mainstream.
- Moreover, every aspect of this era underwent a **three-phase** process: **research and development (R&D), commercialization, and commoditization**.

Parallel & Distributed Computing

- The term parallel computing and distributed computing are **often used interchangeably**, even though they mean **slightly different things**.
- They are based on **systems concepts**, such as **concurrency(**cooperation)**, mutual exclusion, consistency(**uniformity**) in state/memory manipulation, message-passing, and shared-memory models.**
- The term **parallel implies a tightly coupled system**, where as **distributed systems refers to a wider class of system, including those that are tightly coupled**.

Parallel Computing

- More precisely, the term **parallel computing** refers to a model in which **the computation is divided among several processors sharing the same memory**.
- The architecture of **parallel computing system** is often characterized by the **homogeneity of components**: **each processor is of the same type and it has the same capability as the others.**
- The **shared memory** has a **single address space**, which is accessible to all the processors.
- Parallel programs are then **broken down** into **several units** of **execution** that can be allocated to different processors and can **communicate** with each other by means of shared memory.

Parallel Computing Contd…

- Originally **parallel systems** are considered as those architectures that featured **multiple processors sharing the same physical memory** and that were considered a **single computer.**
  - Over time, these **restrictions have been relaxed**, and parallel systems now **include all architectures** that are based on the concept of **shared memory**, whether this is physically present or created with the support of **libraries, specific hardware, and a highly efficient networking infrastructure.**
  - For example: a cluster of which of the nodes are connected through an InfiniBand network and **configured with distributed shared memory system** can be considered as a parallel system.

Distributed Computing

- The term distributed computing **encompasses(**cover**)** any **architecture or system** that allows the **computation** to be **broken down into units** and **executed concurrently on different computing elements**, whether these are **processors on different nodes**, processors on the **same computer**, or cores within the same processor.
- Distributed computing includes a **wider range of systems and applications** than parallel computing and is often considered a more general term.

Distributed Computing Contd…

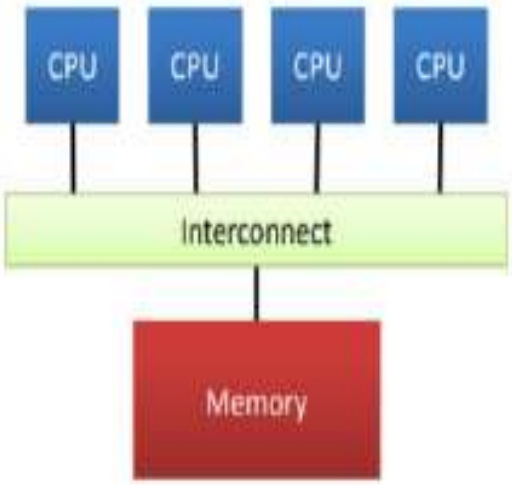
- Even though it is not a rule, the term distributed often implies that the locations of the computing elements are not the same and such elements might be **heterogeneous in terms of hardware and software** features.
- Classic examples of distributed computing systems are
  - Computing Grids
  - Internet Computing Systems

Parallel vs. Distributed Computing

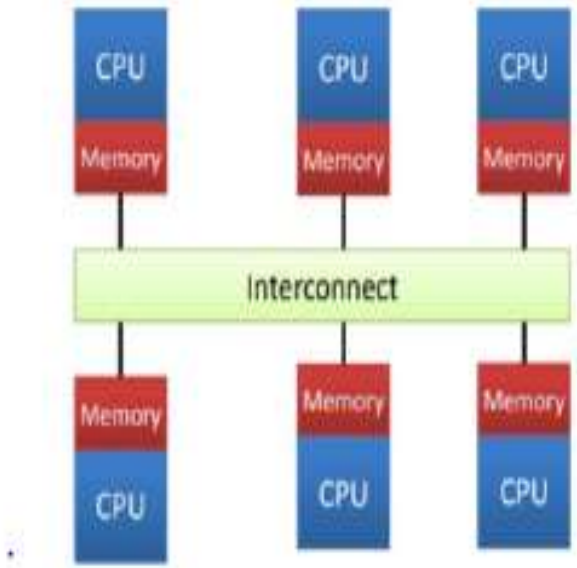
- **Parallel Computing : All processors may have access to a shared memory** to exchange information between processors.
- **Distributed computing : Each processor has its own private** memory (distributed memory) and information is exchanged by passing messages between the processors.

Parallel Systems	Distributed Systems
Tight coupling	Loose coupling
Physical proximity	Server room to Global
Homogeneous	Heterogeneous
Threads & MPI	RPC, Web Services, REST

Shared vs. Distributed Memory



Shared Memory



Distributed Memory

Elements of Parallel Computing

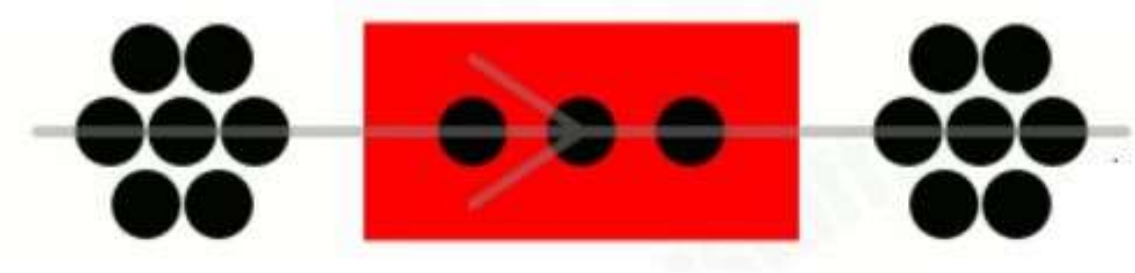
- **Necessity of Parallel Computing**
  - Computational requirements are ever increasing
  - Wastage of hardware resources in serial processing
  - Sequential architectures are reaching their physical limits
  - Processing speed is constrained by the speed of light
  - Density of transistors packaged in a processor is constrained by thermodynamic limitations.
- **Possible Solutions**
  - Connecting multiple processors working in mutual coordination.
  - Developing techniques, architectures, and systems for performing multiple activities in parallel

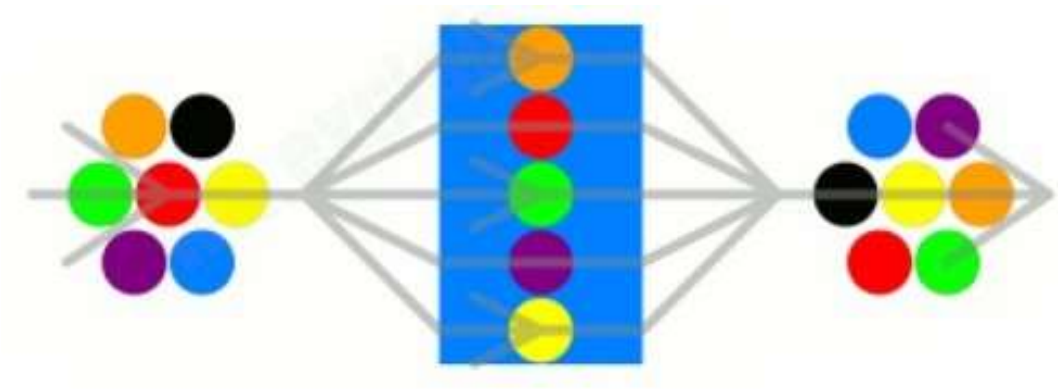
What is Parallel Processing?

- Processing of **multiple tasks** simultaneously on **multiple processors** is called parallel processing.
- The parallel program consists of **multiple active processes** (tasks) simultaneously **solving a given problem**.
- Each task **is divided into multiple subtasks** using **a divide-and-conquer technique**, and each subtask is processed on a **different CPU**.
- Programming on **a multiprocessor system** using the **divide-and-conquer** technique is **called parallel programming**
- Many applications today **require more computing power** than a **traditional sequential computer** can offer.
- Parallel Processing provides a **cost effective solution** to this problem by increasing the number of **CPUs in a computer** and by adding an efficient communication system between them.
- The workload can then be **shared between different processors**. This setup results in higher computing power and performance than a single processor a system offers

Serial vs. Parallel Computing

Serial Computing

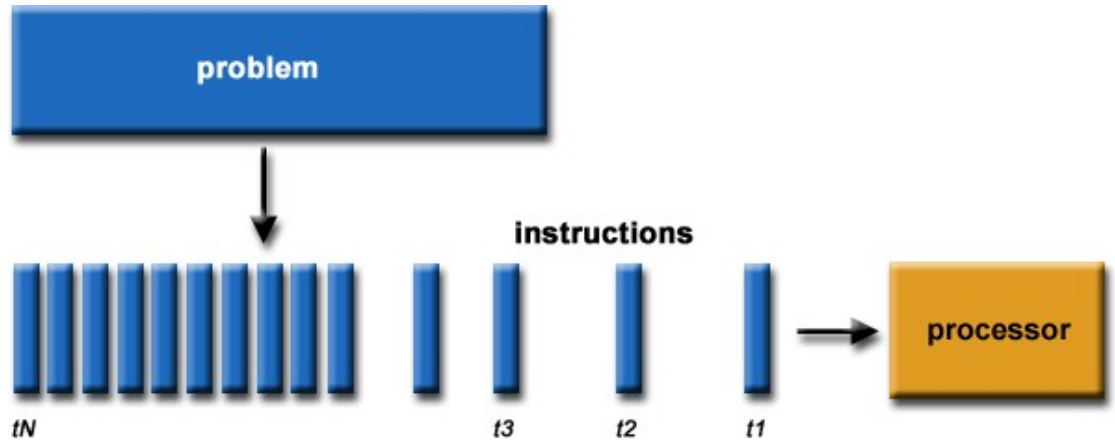


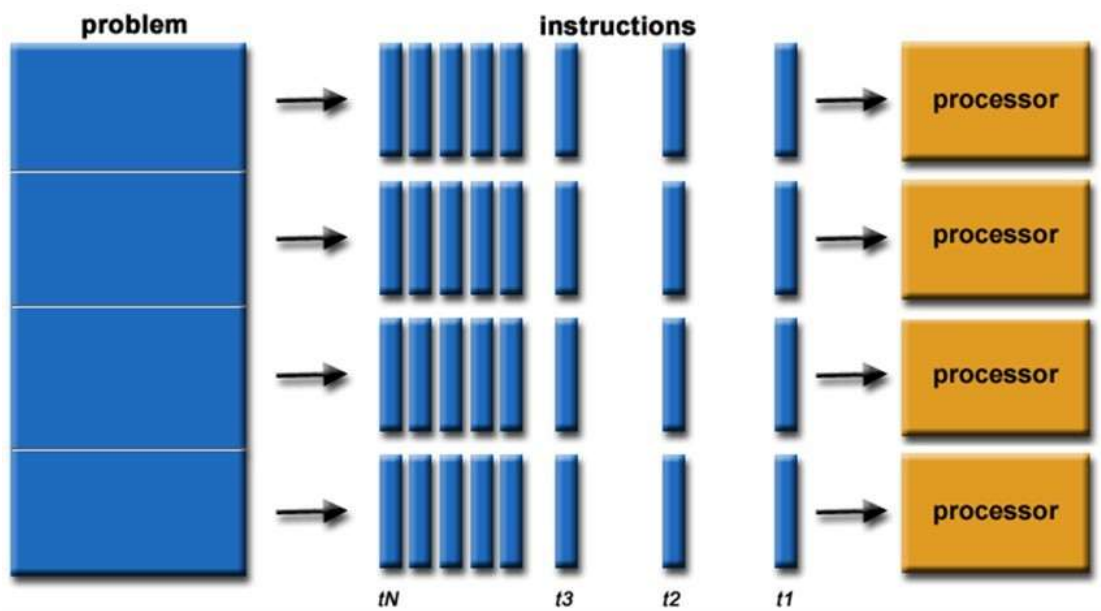


Parallel Computing

Serial vs. Parallel Computing

Serial Computing





Parallel Computing

Parallel Processing influencing factors

- The development of parallel processing is being **influenced by many factors**. The prominent among them include the following:
  - **Computational requirements** are ever increasing in the areas of both **scientific and business computing**. The technical computing problems, which require **high-speed computational power**, are related to
    - life sciences, aerospace, geographical information systems, mechanical design and analysis etc.
  - **Sequential architectures** are reaching **mechanical physical limitations** as they are constrained by the **speed of light and thermodynamics laws.**
    - The **speed** which **sequential CPUs** can operated is reaching **saturation point** (no more vertical growth), and hence an **alternative way** to get **high computation speed** is to connect multiple CPUs (opportunity for horizontal growth).

Parallel Processing influencing factors Contd…

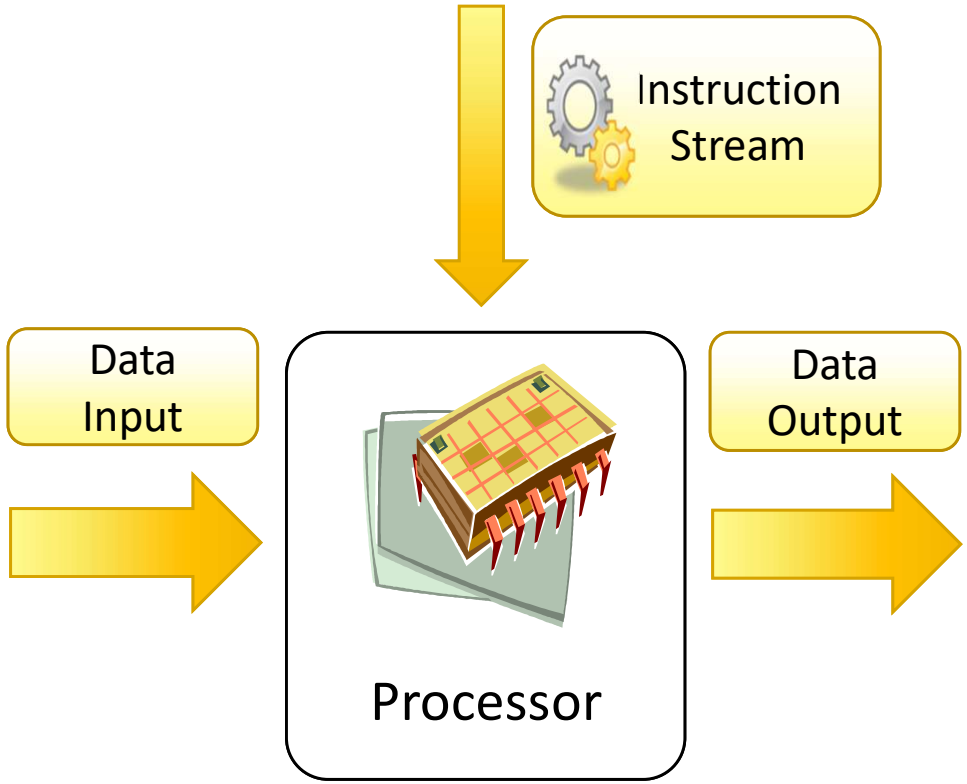
- **Hardware improvements** in pipelining , super scalar, and the like are non scalable and require sophisticated compiler technology.
  - Developing such compiler technology is a difficult task.
- **Vector processing works** well for certain kinds of problems. It is suitable mostly for scientific problems (involving lots of matrix operations) and graphical processing.
  - It is **not useful** for other areas, such as **databases**.
- The technology of parallel processing is mature and can be **exploited commercially**
  - here is already significant R&D work on development tools and environments.
- **Significant development in networking technology** is paving the way for
  - heterogeneous computing.

Hardware architectures for parallel Processing

- The **core elements** of parallel processing are **CPUs.** Based on the **number of instructions and data streams**, that can be processed simultaneously, computing systems are classified into the following four categories:
  - Single-instruction, Single-data (SISD) systems
  - Single-instruction, Multiple-data (SIMD) systems
  - Multiple-instruction, Single-data (MISD) systems
  - Multiple-instruction, Multiple-data (MIMD) systems

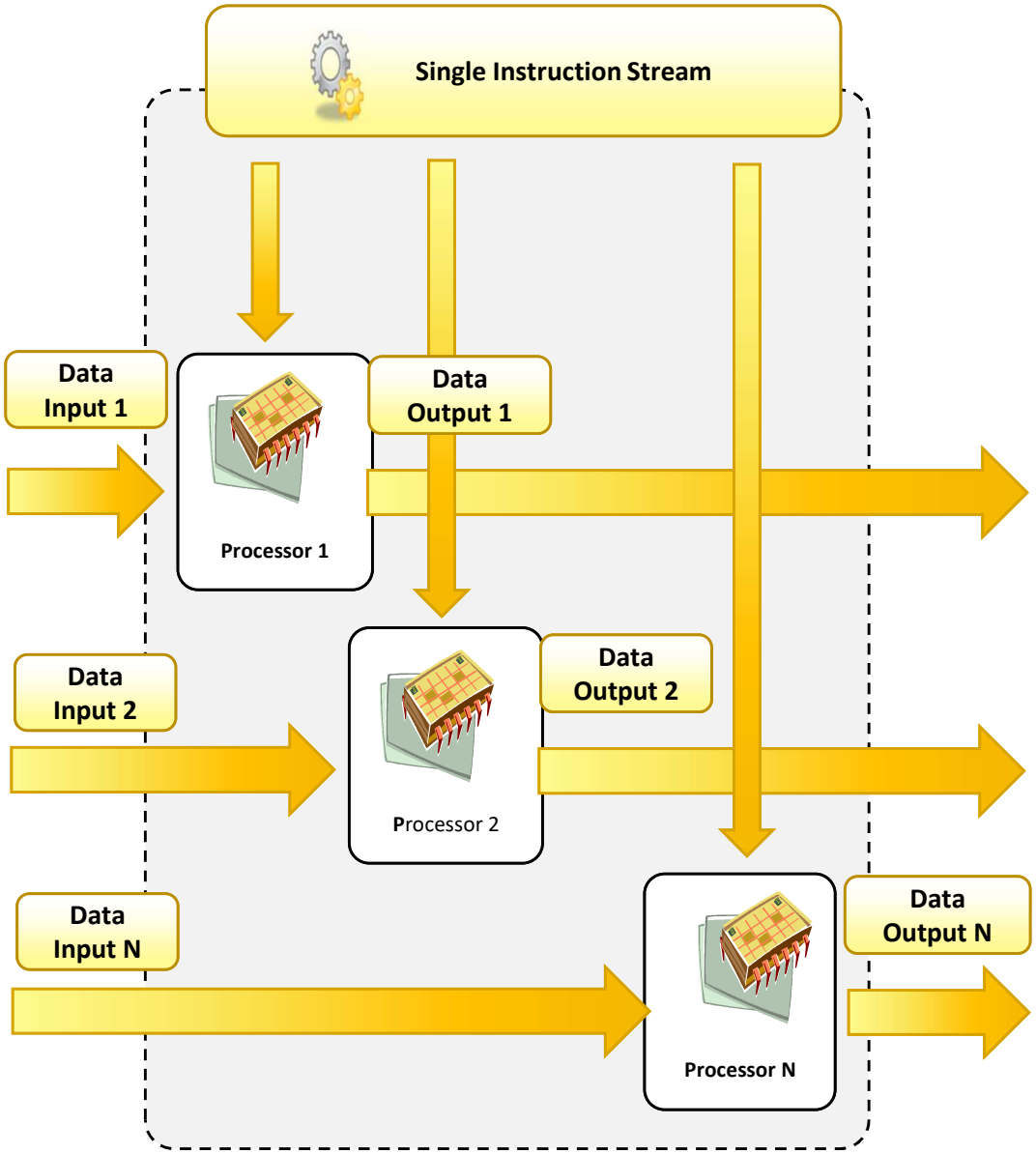
Single – Instruction , Single Data (SISD) systems

- SISD computing system is a **uni-processor** machine capable of **executing a single instruction,** which **operates** on a **single data stream**.
- Machine instructions are **processed sequentially**, hence computers adopting this model are popularly called **sequential computers.**
- Most **conventional computers** are built using SISD model.
- All the **instructions and data** to be **processed** have to be **stored** in primary memory.
- The **speed of processing** element in the SISD model is **limited** by the rate at which the computer can transfer information internally.
- Dominant **representative SISD** systems are IBM PC, Macintosh, and workstations.



Single – Instruction , Multiple Data (SIMD) systems

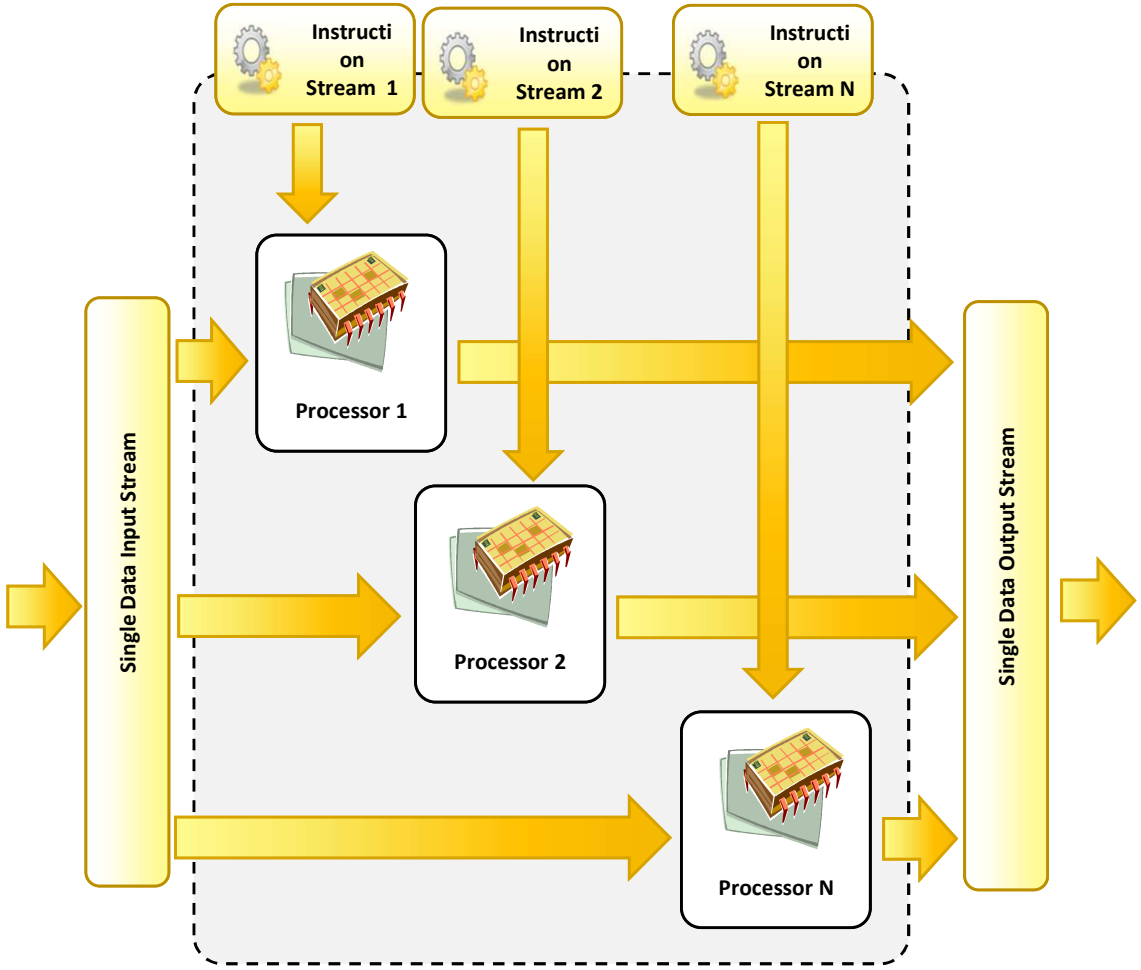
- SIMD computing system is a **multiprocessor machine** capable of executing the **same instruction** on all the CPUs but **operating** on **different data streams.**
- Machines based on this model are well suited for **scientific computing** since they involve lots of **vector and matrix operations.**
- For instance statement **Ci = Ai * Bi,** can be passed to all the **processing elements** (PEs), organized data elements of **vectors A and B** can be **divided into multiple sets** (N- sets for N PE systems), and each PE can process one **data set**.
- Dominant representative SIMD systems are **Cray's Vector processing** machine and Thinking Machines Cm*, and GPGPU accelerators.



Multiple – Instruction , Single Data (MISD) systems

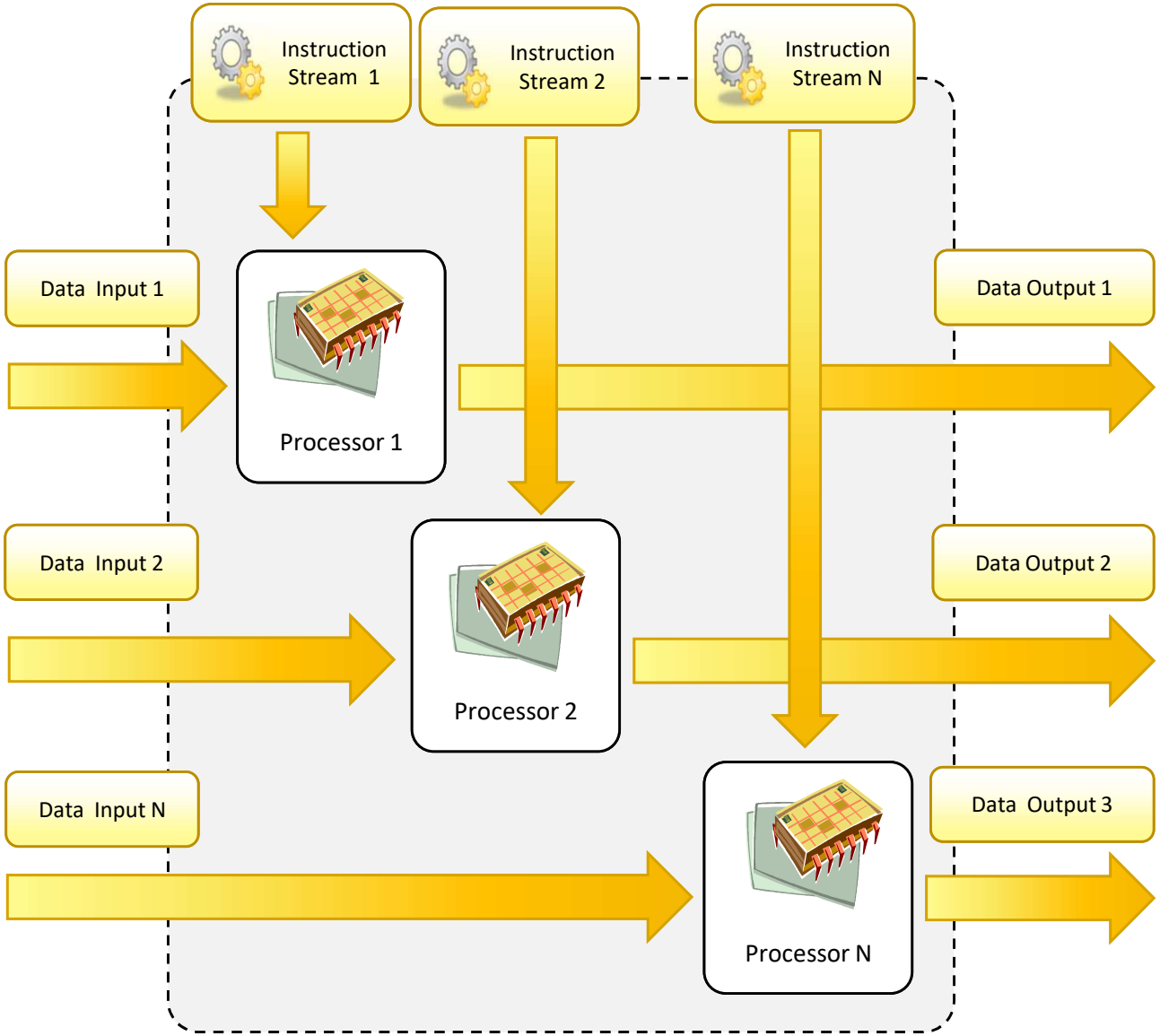
- MISD computing system is a **multi processor machine** capable of **executing different instructions on different PC's** all of them **operating on the same data set.**
- For example

  y = sin(x) + cos(x) + tan(x)
- Machines built using MISD model are **not useful** in most of the applications.
- **Few machines** are built but none of them available commercially.
- This type of systems are more of **an intellectual exercise** than a **practical configuration.**
- MISD is primarily of theoretical importance because **no real system** has been built using the MISD structure.



Multiple – Instruction , Multiple Data (MIMD) systems

- MIMD computing system is a multi processor machine capable of **executing multiple instructions on multiple data sets.**
- Each PE in the MIMD model has **separate instruction and data streams**, hence machines built using this model are well **suited to any kind of application**.
- Unlike SIMD, MISD machine, PEs in MIMD machines **work asynchronously,**
- MIMD machines are broadly categorized into shared-memory MIMD and distributed memory MIMD based on the way PEs are coupled to the main memory.

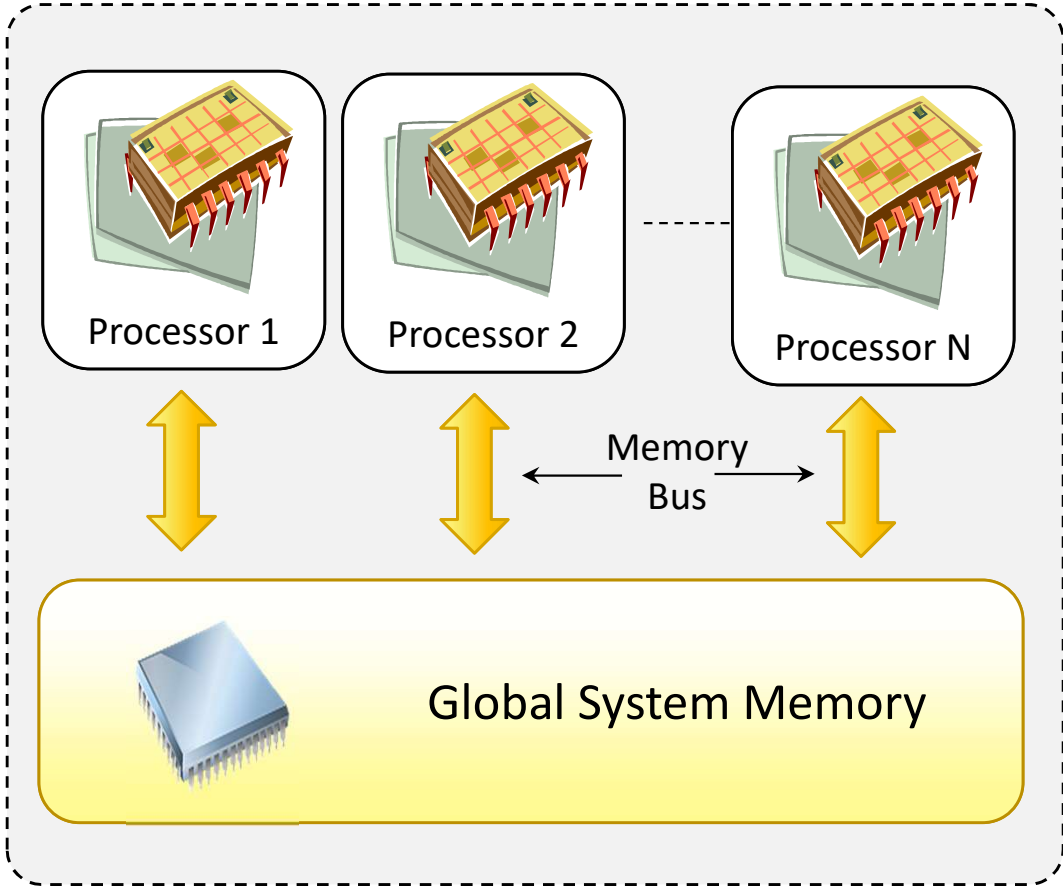


Classification of MIMD Systems

- A multiprocessor machine capable of executing multiple instructions on multiple data set
- Processing elements(PE's) in MIMD machines work asynchronously
- MIMD machines are broadly categorized based on the way PEs are coupled to the main memory i.e.
  - Shared-memory MIMD and
  - Distributed-memory MIMD

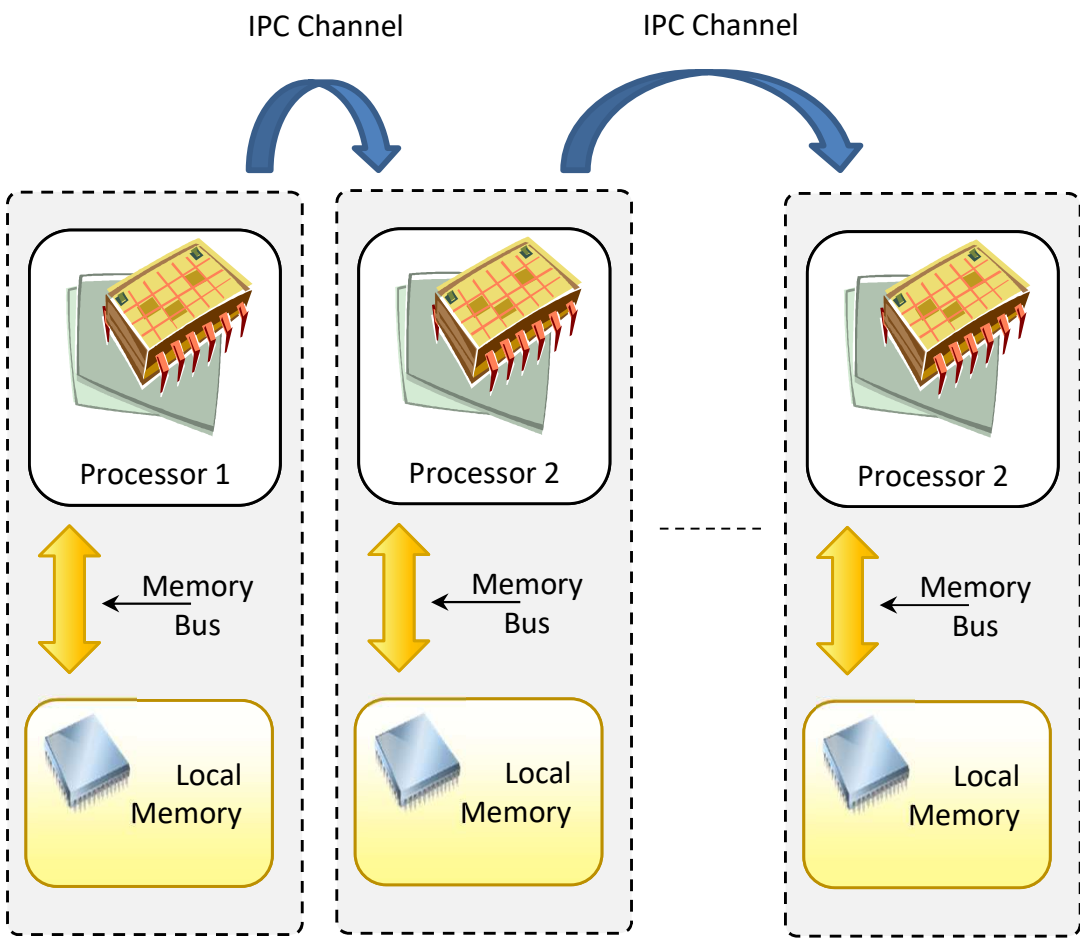
Shared Memory MIMD machines

- All the **PEs** are **connected** to a **single global memory** and they all have access to it.
- **Systems based** on this model are also called **tightly coupled** multi processor systems.
- The **communication** between PEs in this model **takes place** through the **shared memory**.
- **Modification** of the data stored in the global memory by one PE is **visible to all other PEs**.
- Dominant representative shared memory MIMD systems are **silicon graphics machines** and Sun/IBM SMP (Symmetric Multi-Processing).



Distributed Memory MIMD machines

- **All PEs** have **a local memory**. Systems based on this model are also called **loosely coupled** multi processor systems.
- The **communication** between **PEs** in this model **takes place** through the **interconnection network**, the inter process communication channel, or **IPC**.
- The **network connecting** PEs can be **configured** to **tree, mesh, cube**, and so on.
- Each PE operates **asynchronously (**something is not happening at the same time as another thing**)**, and if **communication/ synchronization** among tasks is necessary, they can do so by **exchanging** messages between them.



Shared Vs Distributed MIMD model

- The **shared memory MIMD architecture is easier to program** but is **less tolerant to failures and harder to extend with respect to the distributed memory MIMD model**.
- **Failures**, in a **shared memory** MIMD **affect the entire system**, whereas this is **not the case of the distributed model**, in which each of the **PEs can be easily isolated**.
- Moreover, shared memory MIMD architectures are **less likely to scale** because the addition of **more PEs leads to memory contention.**
- This is a **situation that does not happen** in the case of **distributed memory**, in which each PE has its own memory.
- As a result, distributed memory MIMD architectures are most popular today.

Approaches to Parallel Programming

Parallel Program :

- A **sequential program** is one that **runs** on a **single processor** and has a **single line of control**.
- To make **many processors** collectively **work** on a **single program**, the program must **be divided into smaller independent chunks** so that each processor can **work on separate chunks** of the problem.
- **The program decomposed** in this way is a **parallel program.**

Approaches to Parallel Programming

- A wide **variety of parallel programming** approaches are available. Those are suitable for task-level parallelism.
- Different Parallel Programming Approaches: Here are the following 3 Models
  - Data Parallelism
  - Process Parallelism
  - Farmer-and-worker model

•**Data Parallelism :** Divide-and-conquer technique is used to split data into multiple sets, and each data set is processed on different PEs using the same instruction. This approach is highly suitable to processing on machines based on the SIMD model.

•**Process Parallelism :** Operations with multiple (but distinct) activities can be processed on multiple processors.

•**Farmer-and-worker model :** Based on job distribution approach (One processor is configured as **master** and all other remaining PEs are designated as **slaves**).

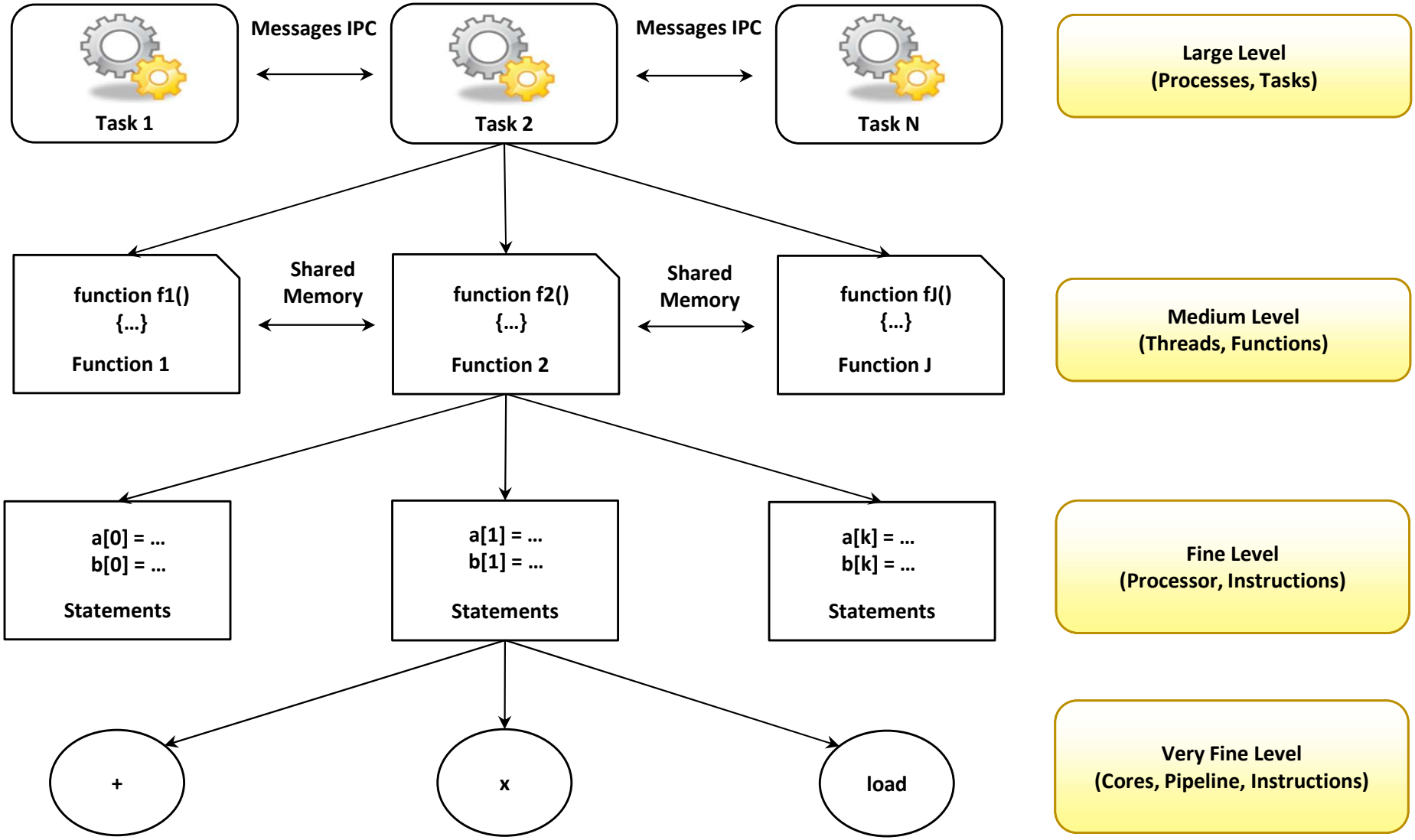
•These approaches can be utilized in different levels of parallelism.

Levels of Parallelism

- Levels of Parallelism are **decided** on the **lumps of code** (group of programmers /grain size) that can be **a potential candidate** (someone who has the skills and experience)of parallelism.
- The table shows the levels of parallelism.
- All these approaches have a common goal
  - To **boost processor efficiency** by hiding latency(delay).
  - To conceal(hide) latency, there must be another **thread ready to run** whenever a lengthy operation occurs.
- The idea is to **execute concurrently two or more single-threaded applications**. Such as compiling, text formatting, database searching, and device simulation.
- parallelism within an application can be detected at several levels:
  - *Large-grain (or task-level)*
  - *Medium-grain (or control-level)*
  - *Fine-grain (data-level)*
  - *Very-fine grain (multiple instruction issue)*

Grain Size	Code Item	Parallelized By
Large	Separate and heavy weight process	Programmer
Medium	Function or procedure	Programmer
Fine	Loop or instruction block	Parallelizing compiler
Very Fine	Instruction	Processor

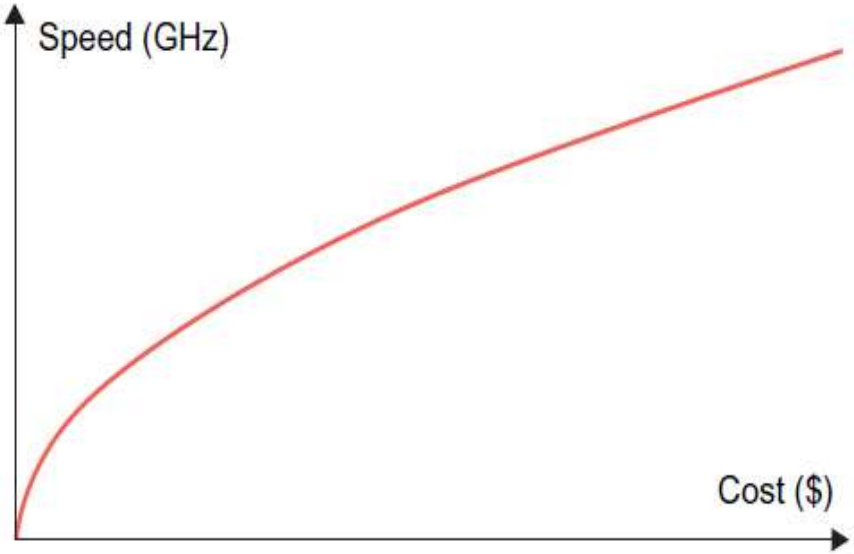
Levels of Parallelism



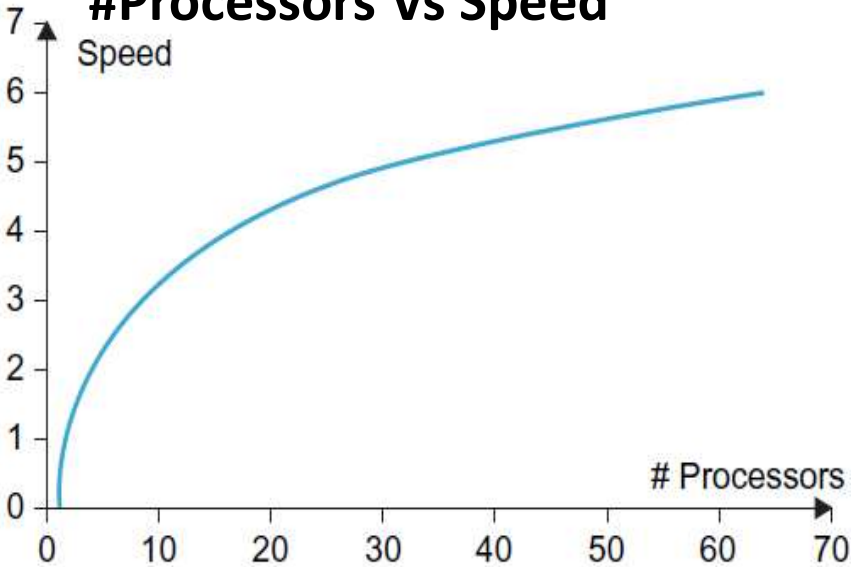
Laws of Caution

- The laws of caution in parallel computing are a **set of principles** that **help explain why adding more processors doesn't always result in a significant speed increase**.
- **Studying** how much an application or a software system can **gain from parallelism**.
- In particular what need to keep in mind is **that parallelism is used to perform multiple activities together** so that the system can **increase** its **throughput or its speed**.
- But the relations that control the increment of speed are not linear.
- For example: for a given n processors, the user expects speed to be increase by in times. This is an ideal situation, but it rarely happens because of the communication overhead.
- Here two important guidelines to take into account.
  - **Speed of computation is proportional to the square root of the system cost**; they never increase linearly. Therefore, the faster a system becomes, the more expensive it is to increase its speed
  - **Speed by a paralle**l computer increases as the **logarithm** of the number of processors (i.e. **y=k*log(N)).**

Speed Vs Cost



#Processors Vs Speed



Elements of Distributed Computing

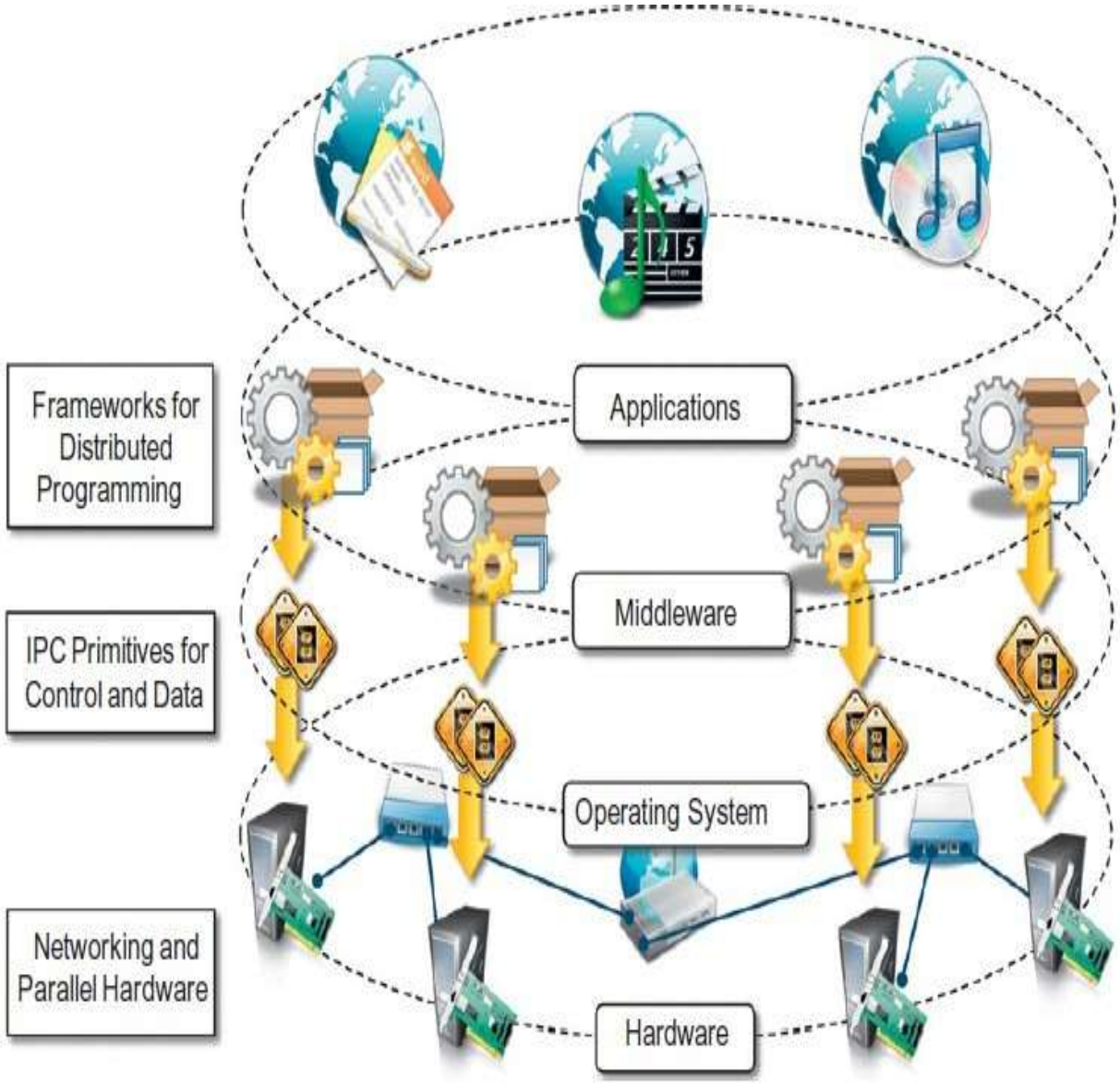
- In the previous section **we discussed techniques and architectures** that allow introduction of **parallelism** within **a single machine or system** and **how parallelism operates at different levels of the computing stack**.
- Here **extend** these concepts and explore how **multiple activities** can be performed by leveraging(improve) systems composed of **multiple heterogeneous** machines and systems.
- We discuss what is generally referred to as distributed computing and more precisely introduce the most common guidelines and patterns for implementing distributed computing systems from the perspective of the software designer.

General concepts and definitions

- **Distributed computing** studies **the models, architectures, and algorithms** used **for building and managing distributed systems**.
- *"A distributed system is a collection of independent computers that appears to its users as a single coherent system."*
- This **definition** is general enough to include various types of distributed computing systems that are **especially focused on unified usage and aggregation of distributed resources.**
- **Communications is another fundamental aspect** of distributed computing. Since distributed systems **are composed of more than one computer that collaborate together**, it is **necessary** to provide some sort of **data and information exchange between them**, which generally occurs **through the network.**
- *"A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages."*
- The components of a distributed system communicate with some sort of ***message passing.***Message passing encompasses several communication models.

Components of a Distributed System

- A distributed system is the result of the **interaction of several components** that traverse the entire computing stack from **hardware to software.**
- Fig. provides an overview of the **different layers** that are involved in providing the services of a distributed system.



A layered view of Distributed System

Components of a Distributed System

Bottom Layer : it is **combination of 2 layers** those are **hardware and Operating System**

- **Physical Infrastructure**: computer and network hardware
- **Operating System** : the operating system that provides the **basic services** for **IPC, Process Scheduling, File system management.**
- these **two layers become the platform on top** of which specialized software is deployed to turn a set of networked computers into a distributed system.

Middleware Layer :

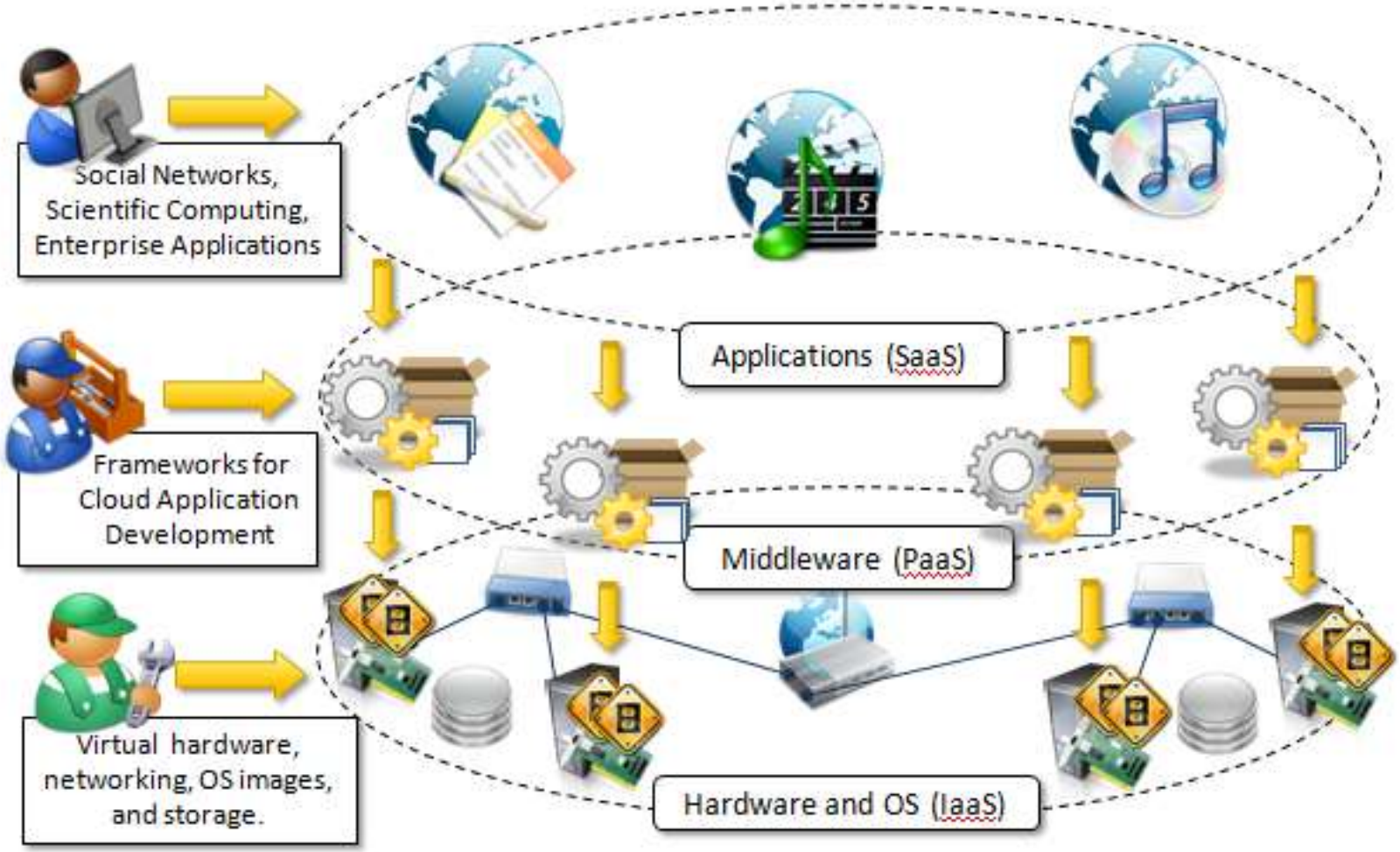
- Leverages (improve) services to **build a uniform environment** for the **development and deployment** of distributed applications.
- Supports programming paradigms for distributed systems
- Provides a **uniform interface** to developers (independent from the underlying OS and hiding the heterogeneities of the bottom layers)

Application Layer :

- Provides GUI's designed and developed to use the middleware.

Components of a Distributed System example

- Fig shows an example on how the general reference architecture of a distributed system is contextualized in the case of Cloud computing system.



Components of a Distributed System example

Bottom Layer :

- Make up the physical infrastructure of one or more datacenters, where racks of servers are deployed and connected together through high speed connectivity.
- OS manages this infrastructure (machine and network management

Middleware Layer :

- Core logic is implemented in the middleware
- Provides a customizable runtime environment for applications

Application Layer :

- Services are offered through Web 2.0-compliant interfaces.
- Facilities catered ranges from virtual infrastructure building and deployment to application development and runtime environments.

Architectural Styles for Distributed Computing

- *"Architectural styles helps in understanding the interactions among different components (SW & HW) and their responsibilities in a distributed computing environment (across* multiple machines.*)".*
- *"Architectural styles are mainly used to determine the vocabulary of components and connectors that are used as instances of the style together with a set of constraints on how they can be combined".*
- Architectural styles for distributed systems can be categorized into two major classes :
  - Software architectural styles --Logical organization of the software
  - System architectural styles-- Physical organization of distributed software

Architectural Styles for Distributed Computing

- 1.Components and Connectors
- 2.Software architectural styles
- (a) Data-Centered Architectures
- i.Knowledge Sources.
- ii.Blackboard.
- iii.Control.
- (b) Data-Flow Architectures.
- i.Batch-Sequential Style.
- ii.Pipe-and-Filter Style.
- (c) Virtual Machine Architectures.
- i.Rule-Based Style.
- ii.Interpreter Style.
- (d) Call and Return Architectures.
- i.Top-Down Style.
- ii.Object-Oriented Style.
- iii.Layered Style.
- (e) Architectural Styles based on Independent Components.
- i.Communicating Processes.
- ii.Event Systems.
- 3.System Architectural Styles
- (a) Client-Server.
- i.Thin-client Model.
- ii.Fat-client Model.
- Two-tier Architecture.
- Three-tier Architecture/N-tier Architecture.
- (b) Peer-to-Peer.

Architectural Styles for Distributed Computing

we clarify what we intend(planned) for *components* and *connectors*, since these are the basic building blocks with which architectural styles are defined.

Components & Connectors

•**Component** represents **a unit of software** that **encapsulates a function** or a **feature** of the system.

•**Examples** of components can **be programs, objects, processes, pipes, and filters.**

•**Connector** is a **communication mechanism** that allows **cooperation and coordination** among **different components**.

•They are **not encapsulated in a single entity** and are implemented in a distributed manner over many system components.

Software Architectural Styles

- Software architectural styles are based on **the logical arrangement** of software components.
- Provide **an intuitive(**understanding**) view** of the whole system, despite its physical deployment.

Software Architectural Styles

Category	Most Common Architectural Styles
Data-centered	Repository Blackboard
Data flow	Pipe and filter Batch sequential
Virtual machine	Rule-based system Interpreter
Call and return	Main program and subroutine call/top-down systems Object-oriented systems Layered systems
Independent components	Communicating processes Event systems

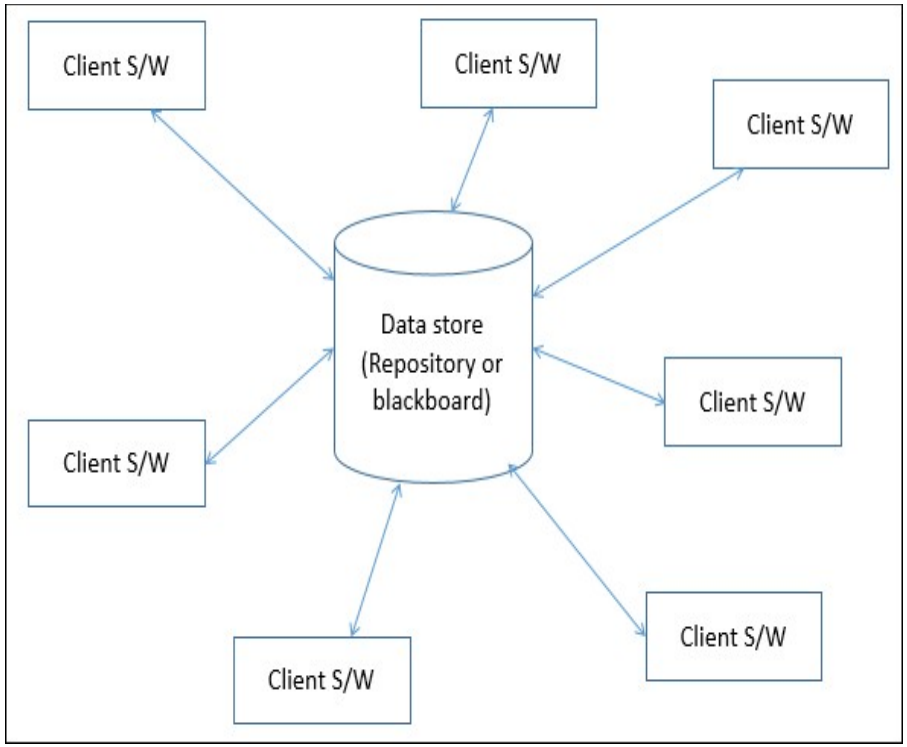
Software Architectural Styles

Data Centered Architecture :

- Data is **centralized and accessed frequently** by other components, which modify data.
- Achieving **integrality of data** is most important
- Different components **communicate** through **shared data repositories**
- Example: **Database** architecture and **Web** architecture

Types of Components

- A **central data** structure or data store
- A **data accessor** or collection of different components operate on the central data store and perform computations, and put back the results.



Software Architectural Styles

Data Centered Architecture :

- **Communication** between the **data accessors** is only through the **data store**.
- **The data** is the only means of communication among **clients.**
- **Flow of control differentiates** the architecture into **two categories :-**

- **Repository architecture Style :** The **data store is passive** and **the clients** (software components or agents) **are active**, which control the flow logic.
- **Blackboard architecture Style :** The **data store is active** and **the clients are passive**, and the logical flow is determined by the current data status of the data store.

Software Architectural Styles

Data Centered Architecture :

Repository Architectural style

- The **repository architectural style** is the most relevant **reference model** in this category. It is characterized by **two main components** – **the central data structure**, which represents the current state of the system, and a collection of independent component, which operate on the central data.
- In particular **repository based architectures differentiate and specialize further into subcategories according to the choice of control discipline to apply for the shared data structure. Of particular interest are databases and blackboard systems.**
- In the repository systems, **the dynamics of the system is controlled by independent components, which by issuing an operation on the central repository, trigger the selection of specific processes that operate on data.**

Software Architectural Styles

Data Centered Architecture :

- **Black board Architectural Style**
- The black board architectural style is characterized by three main components:
  - **Knowledge sources** : These are entities that update the knowledge base that is maintained in the black board.
  - **Blackboard** : This represents the **data structure** that is shared among the knowledge sources and stores the knowledge base of the application.
  - **Control:** The control is the **collection of triggers and procedures** that govern the interaction with the blackboard and update the status of the knowledge base.

Software Architectural Styles

Data Flow Architecture

- The whole software system is viewed as a **series of transformations on consecutive pieces of input data**, where data and operations are independent of each other.
- Data enters into the system and then flows through the **modules one at a time until they are assigned to some final destination** (output or a data store).
- **Connections** between the components may be implemented as **I/O stream, I/O buffers, piped, or other** types of connections.
- Suitable for applications involving a **well-defined series of independent data transformations** (computations) on orderly **defined input and output.**

Software Architectural Styles

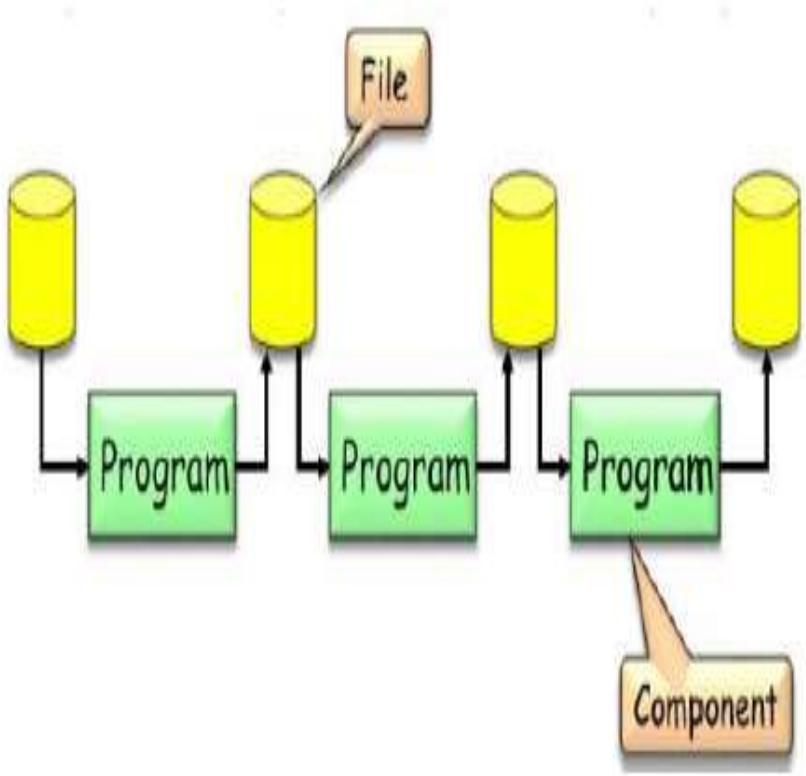
- **Data Flow Architecture**
- Data Flow Architecture approaches can be classified based on execution of sequences between modules
- Batch Sequential
- Pipe and Filter
- Process Control

Software Architectural Styles

Data Flow Architecture

Batch Sequential :

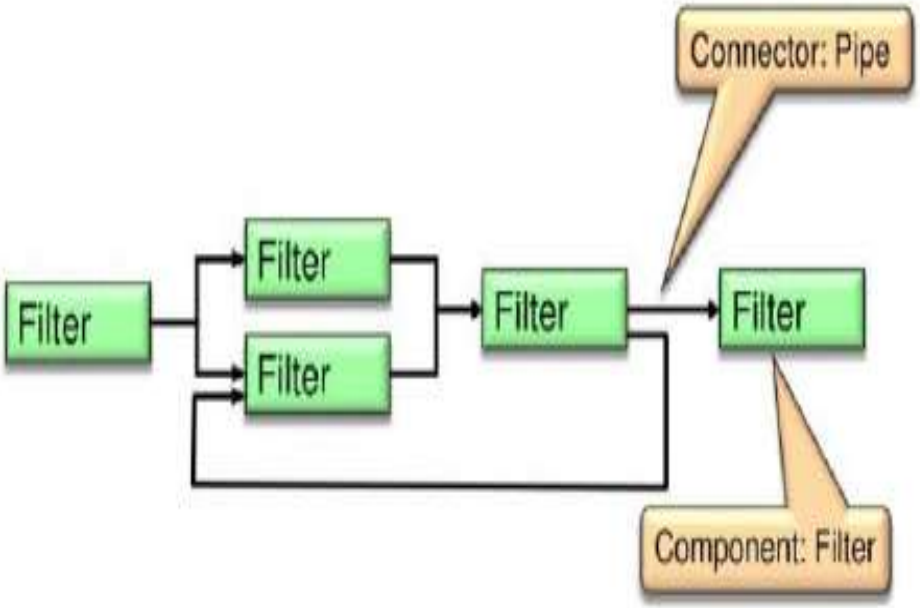
- A **sequential data** processing model
- Separate programs are **executed in order** and the **data is passed as an aggregate** from one program to the next.
- Provides **simpler divisions on subsystems** and each can be **an independent program working** on input data and produces output data.
- **Features**
- Components are independent programs
- Connectors are media (typically files)
- Each step runs to completion before next step begins
- **Disadvantages**
- Provides high latency and low throughput.
- Does not provide concurrency and interactive interface.



Software Architectural Styles

Data Flow Architect

- **Pipe and Filter:**
- A variation of the batch sequential style **for expressing the activity of a software system as sequence of data transformations.**
- Filter is an **independent entity** which perform transformation on data and process the input they receive.
- Pipes **serve as connectors** for the stream of data being transformed, each connected to the next component in the pipeline.
- **Features**
- Ensures loose and flexible coupling of components, filters
- Supports parallel processing
- Each filter can be called and used over and over again
- **Disadvantages**
- Reduced performance due to excessive computational overheads.
- Unsuitable for an interactive system
- May not be appropriate for long running computations.



Software Architectural Styles

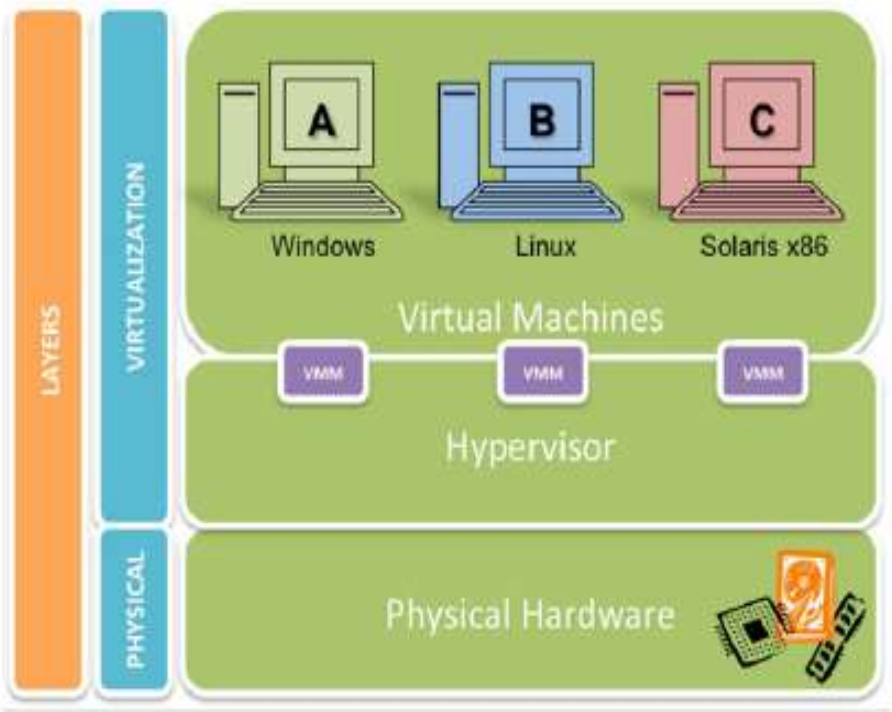
Comparison between Batch Sequential and Pipe-and-Filter Styles

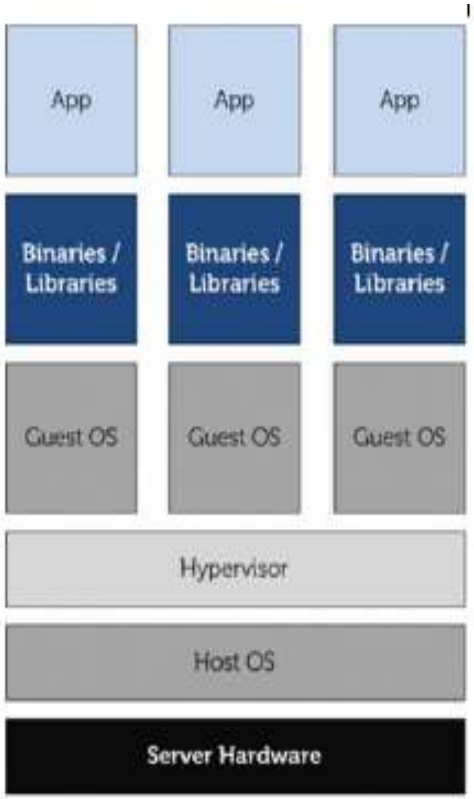
Batch Sequential	Pipe-and-Filter
Coarse(roughf) grained	File grained
High latency	Reduced latency due to the incremental processing of input
External access to input	Localized input
No concurrency	Concurrency possible
Non interactive	Interactivity awkward but possible

Software Architectural Styles

Virtual Machine architectures

- The virtual machine class of architectural styles is Characterized by the presence of an **abstract execution environment** (VM)
- Simulates features that are not available in the H/W or S/W
- Applications are implemented on top of this virtualization layer





Software Architectural Styles

Virtual machine architectures contd…

- Popular examples with in this category are rule based systems, interpreters, and command language processors.
- **Rule-Based Style:** This architecture is characterized by representing the **abstract execution environment** as **an inference engine**. Programs are expressed in the form of rules or predicates that hold true.
- The **input data** for applications is generally **represented by a set of** assertions or facts that the inference engine uses to activate rules or to apply predicates, thus transforming data. The examples of rule-based systems can be found in the networking domain**: Network Intrusion Detection Systems (NIDS)** often rely on a set of rules to identify abnormal behaviors connected to possible intrusion in computing systems.
- **Interpreter Style:** The interpretation of the pseudo program constitutes the execution of the program itself.
- Systems modelled according to this style exhibit four main components: the interpretation engine that executes the core activity of this style, an internal memory that contains the pseudo-code to be interpreted, a representation of the current state of the engine, and a representation of the current state of the program being executed.

Software Architectural Styles

- **Call and return architectures**
- This identifies all systems that are organized into components mostly connected together by method calls.
- The activity of systems modeled in this way is characterized by a chain of method calls whose overall execution and composition identify the execution one or more operations.
- There are three categories in this
  - Top down Style : developed with imperative programming
  - Object Oriented Style: Object programming models
  - Layered Style: provides the implementation in different levels of abstraction of the system.

Software Architectural Styles

Architectural Styles based on Independent Components.

- This class of architectural styles model systems in terms of independent components, having their own life cycle, which interact to each other in order to perform their activities. There are two major categories within this class, which differentiate in the way the interaction among components is managed.
- **Communicating Processes**. In this architectural style, components are represented by independent processes that leverage inter-process communication (IPC) facilities for coordination management.
- **Event Systems.** In this architectural style, the components of the system are loosely coupled and connected. In addition to exposing operation for data and state manipulation, each component also publishes (or announces) a collection of events that other components can register with.

System Architectural Styles

- Deals with physical organization of components and processes over a distributed infrastructure
- • Provide a set of reference models for the deployment of distributed systems
- • Fundamental reference styles
- • Client/server
- • Peer-to-Peer.

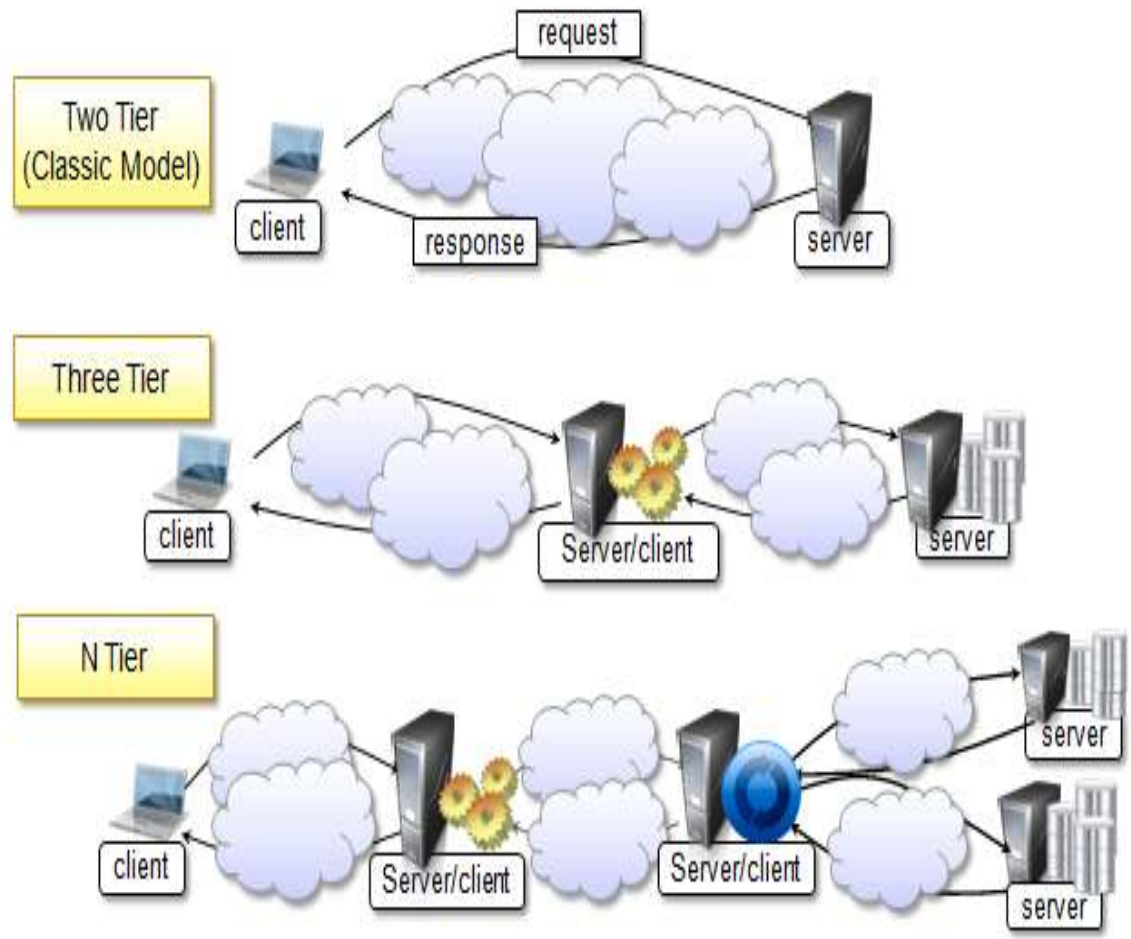
System Architectural Styles

Client / Server

- The information and the services of interest can be centralized and accessed through a single access point : the server.
- Multiple clients are interested in such services and the server must be appropriately designed to efficiently serve requests coming from different clients.
- This consideration has implications on both client design and server design. For the client design, we identify two major models:
- **Thin-client Model**. In this model, the load of data processing and transformation is put on the server side, and the client has a light implementation that is mostly concerned with retrieving and returning the data it is being asked for, with no considerable further processing.
- **Fat-client Model**. In this model, the client component is also responsible for processing and transforming the data before returning it back to the user, while the server features a relatively light implementation mostly concerned with the management of access to the data.

System Architectural Styles

- The mapping between the conceptual layers and their physical implementation in modules and components allows differentiating among several types of architectures, which go under the name of multi-tiered architectures. Two major classes exist:
- **Two-tier Architecture**. This architecture partitions the systems into two tiers, which are located one in the client component and the other on the server.
- **Three-tier Architecture/N-tier Architecture**. The three-tier architecture separates the presentation of data, the application logic, and the data storage into three tiers.



System Architectural Styles

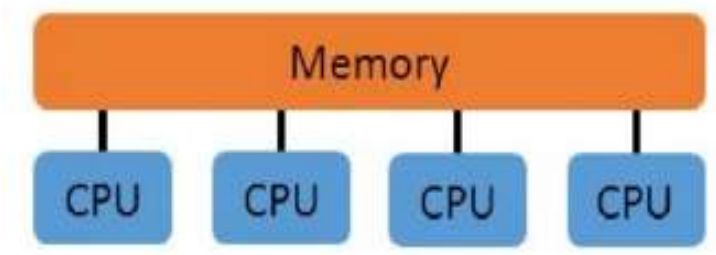
- **Peer- to – Peer**
- Symmetric architectures in which all the components, called peers, play the same role and incorporate both client and server capabilities of the client/server model.
- More precisely, each peer acts as a server when it processes requests from other peers and as a client when it issues requests to other peers.

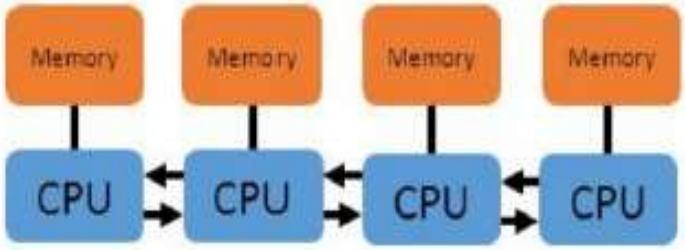
Models for Inter process Communication

- **Inter Process Communication**
- Process is the instance of a computer program that is being executed.
- • A process can be classified as
- • Independent process--Not affected by the execution of other processes
- • Co--operating process--Affected by other executing processes
- • Inter process communication (**IPC) is a mechanism which allows**
- **processes to communicate with each other and synchronize** their **actions.**
- • IPC is a method of co--operation between the processes for problem solving.
- • Processes can communicate with each other through
- • Shared Memory
- • Message Passing
- • Remote Procedure Call (RPC)

Models for Inter process Communication

- **Shared Memory vs. Message Passing**
- **Shared Memory**
- •Implicit communication via memory operations (load/store/lock)
- •Global address space
- **Message Passing**
- •Communicate data among a set of processors without the need for a global memory
- •Each process has its own local memory and communicate with others using message passing



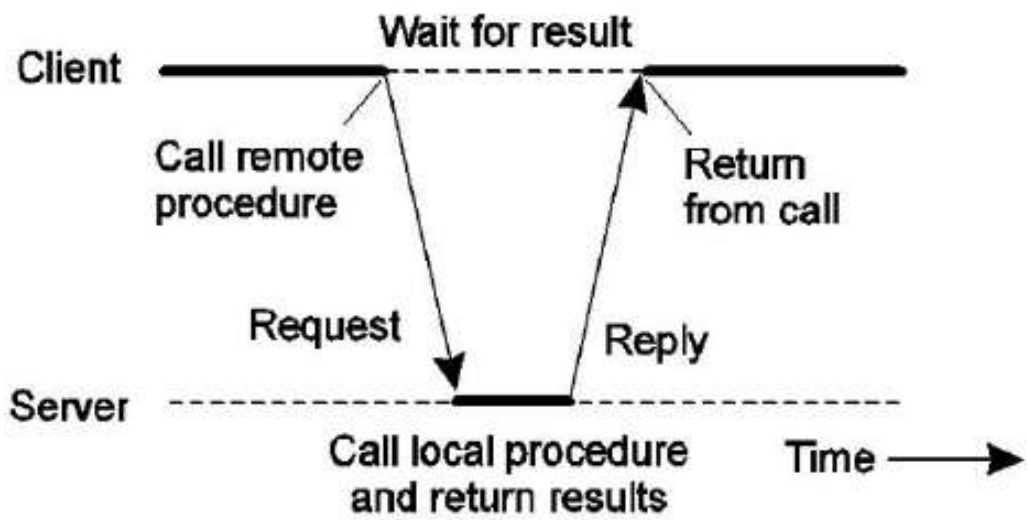


Models for Inter process Communication

- **Remote Procedure Call (RPC)**
- •Protocol for constructing distributed client-server based applications
- •An abstraction of inter--process communication
- •RPC runtime takes care of the network difference between the processes
- •Called procedure need not exist in the same address space as the calling procedure
- •Processes may be on the same system, or they may be on different systems with a network connecting them

Models for Inter process Communication

- **Basic Remote Procedure Call Operation**



Process communication between caller & callee can be hidden using procedure call mechanism

Models for Inter process Communication

Message based Communication in IPC

- Message-based communication model refers to various model for inter-process communication, which is based on the data streaming abstraction
- • Major distributed programming models that uses “message based communication model”
- • Message Passing
- • Remote Procedure Call
- • Distributed Objects
- • Active Objects
- • Web Services

Models for Inter process Communication

- **Message based Communication in IPC**
- **Message Passing**
- • The concept of message is the main abstraction of the model
- • Entities exchanging data and information explicitly encode in the form of message
- • **Remote Procedure Call**
- • Explores the possibilities of remote procedural call (execution of program in remote processes)
- • **Distributed Objects**
- • Implementation of RPC model for the Object-oriented model
- • Contextualizes remote invocation of methods extended by objects
- • **Active Objects**
- • The Active object pattern's goal is to separate the method calling from method execution.
- **Web Services**
- • Provides an implementation of the RPC concept over HTTP
- • Allows interaction of components developed with different technologies
- **Summary:**
- •Message is a fundamental abstraction of IPC, and is used either explicitly or implicitly
- •Principal use of message passing is to define interaction protocols among distributed components for coordinating data exchange activity

Models for Inter process Communication

Models for message-based communication

- Constitutes a fundamental block for several distributed programming paradigms
- • Important to understand how messages are exchanged and among how many distributed components
- • Client server model has been identified as reference model for interaction in most cases
- • Variations of the client server model allow for different interaction patterns
- • Point-to-point message model:This model organizes the communication among single components.
- • Publish-and-subscribe message mode:This model introduces a different strategy which is based on notifi cation among components. There are two major roles:
  - Push Strategy
  - Pull Strategy
- • Request-reply message model:The request-reply message model identifi es all those communication models, where for each of the message sent by a process, there is a reply.

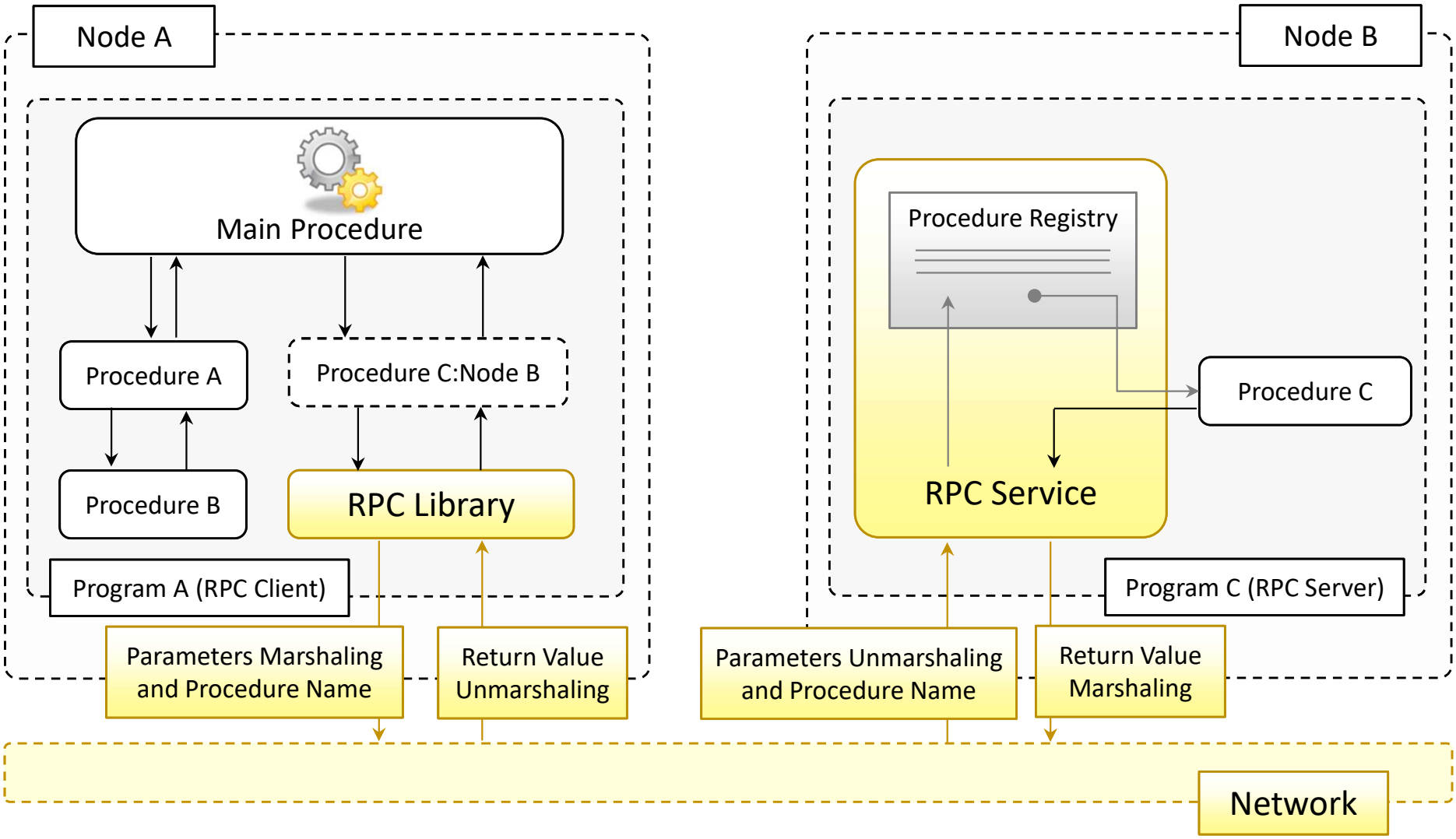
Technologies for distributed computing

- Relevant technologies that provide concrete implementations of interaction models which mostly rely on message-based communication
- • Remote Procedure Call
- • Distributed Object Framework
- • Examples of Distributed Object Framework
- • Common Object Request Broker Architecture(CORBA)
- • Distributed component object model(DCOM/COM1)
- • Java Remote Method Invocation (RMI)
- • .NET remoting

Technologies for distributed computing

- Remote Procedure Call (RPC)
  - RPC is the fundamental abstraction enabling the execution procedures on clients' request.
  - RPC allows extending the concept of a procedure call beyond the boundaries of a process and a single memory address space.
  - The called procedure and calling procedure may be on the same system or they may be on different systems.
  - The important aspect of RPC is marshalling and unmarshalling.

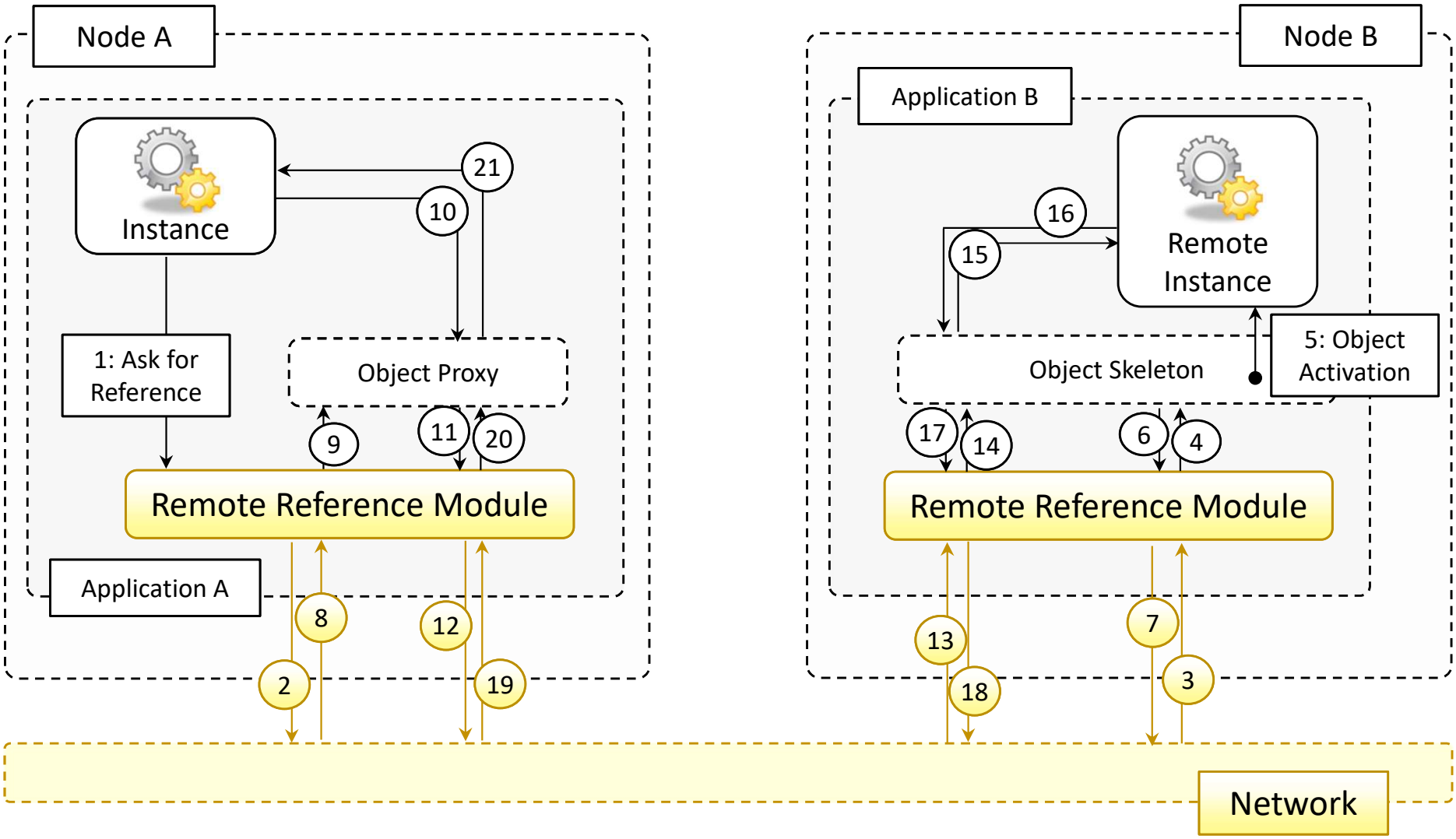
Remote Procedure Call (RPC)



Technologies for distributed computing

- Distributed Object Frameworks
  - Extend object-oriented programming systems by allowing objects to be distributed across a heterogeneous network and provide facilities so that they can be coherently act as though they were in the same address space.

Distributed Object Frameworks



Examples of distributed Object frameworks

- Common Object Request Broker Architecture (CORBA): cross platform and cross language interoperability among distributed components.
- Distributed Component Object Model (DCOM/COM+) : Microsoft technology for distributed object programming before the introduction of .NET technology.
- Java Remote Method Invocation (RMI): technology provided by Java for enabling RPC among distributed Java objects.
- .NET Remoting: IPC among .NET applications, a uniform platform for accessing remote objects from within any application developed in any of the languages supported by .NET.

Technologies for distributed computing

Service Oriented Computing

- Organizes distributed systems in terms of services representing major abstraction for building systems
- • Service orientation expresses applications and software systems as aggregations of services that are coordinated within a service-oriented architecture(SOA)
- • Web services are the de facto approach for developing SOA
- • Web services enables cloud computing systems leverage the Internet as the main interaction channel between users and the system

Technologies for distributed computing

Service Oriented Computing

- **What is a service?**
- • A service encapsulates a software component that provides a set of coherent and related functionalities that can be reused and integrated into bigger and more complex applications
- • Major characteristics that identify a service
- • **Boundaries are explicit--**Application composed of services that are spread across different domains, trust authorities, and execution environments
- • **Services are autonomous--**Not designed to be part of a specific system, but can be integrated in several S/W systems at the same time
- • **Services share schema and contracts, not class or interface definitions**
- • Not expressed in terms of classes or interfaces
- • Define themselves in terms of schemas and contracts
- • **Services compatibility is determined based on policy—**Service orientation separates structural compatibility from semantic compatibility

Technologies for distributed computing

Service Oriented Architecture (SOA)

- SOA organizes a software system into a collection of interacting services
- • Encompasses a set of design principles for facilitating system development
- • Provides a means for integrating components into a coherent and decentralized system
- • Service provider and service consumer are two major roles within SOA

Technologies for distributed computing

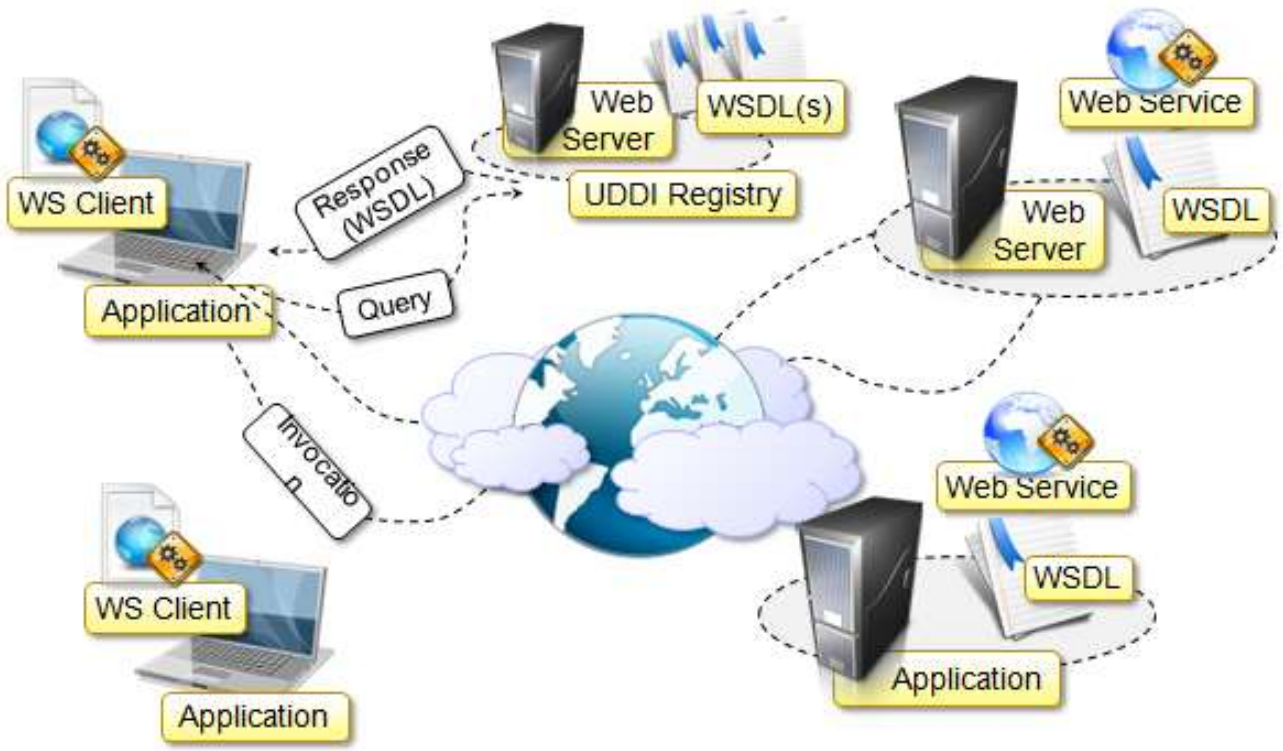
Service Oriented Architecture (SOA)

- A reference model for architecting several software systems, especially enterprise business applications and other systems
- • Guiding principles which characterize SOA platforms
- • Standardized Service Contract
- • Loose Coupling
- • Abstraction
- • Reusability
- • Autonomy
- • Lack of State
- • Discoverability
- • Composability

Technologies for distributed computing

Service Oriented Architecture (SOA)

- Web Services
- Prominent technology for implementing SOA systems and applications
- • Leverage internet technologies and standards for building distributed systems
- • Allow interoperability across different platforms and programming languages
- • Characteristics that makes web services the technology of choice for SOA
- • Allow for interoperability across different platforms and programming languages
- • Based on well-known and vendor-independent standards such as HTTP, SOAP, XML, and WSDL
- • Provide an intuitive and simple way to connect heterogeneous software systems
- • Provide features required by enterprise business applications to be used in an industrial environment

Technologies for distributed computing

- Web Service Interaction